A HANDS-ON TUTORIAL FOR .NET DEVELOPERS

RAG from Scratch in .NET

Build a retrieval-augmented generation system with C#, local open-source models and PostgreSQL. Based on the course *Hands-On AI: Build a RAG Model from Scratch with Open Source*, rebuilt step by step for .NET.

1 Understanding RAG

2 LLM and data

3 Vector database

4 Full pipeline

5 Beyond the course

.NET 10 · C# · Ollama · Granite 4.1 · Qwen3-Embedding · PostgreSQL 16 + pgvector · EF Core · Docker Desktop

Every tool in this book is free, and every model and library is open source.

Contents

START HERE **The Plan**

- What you will build
- The phases at a glance
- How each phase is organised
- The technology stack
- Your hardware
- The solution you will build

PHASE 1 **Understanding RAG**

- 1.1 The problem RAG solves
- 1.2 The big picture
- 1.3 How an LLM writes text
- 1.4 Tokens: how text becomes numbers
- 1.5 Embeddings: meaning as numbers
- 1.6 Measuring similarity: cosine
- 1.7 Chunking: one idea per vector
- 1.8 Vector databases
- 1.9 Prompts: putting it together
- 1.10 Mapping the concepts to the code

PHASE 2 **Setting Up Your LLM and Data**

- 2.1 What runs where
- 2.2 Install the tools
- 2.3 Install Ollama and the models
- 2.4 The Modelfile: a custom model
- 2.5 Start PostgreSQL with pgvector
- 2.6 Create the .NET solution
- 2.7 Settings in one place
- 2.8 Talking to the models from C
- 2.9 Cosine similarity by hand
- 2.10 The console app
- 2.11 Build the corpus from Wikipedia
- 2.12 Your own documents: PDF, Word, PowerPoint

PHASE 3 **The Vector Database**

- 3.1 Embeddings in practice
- 3.2 The table
- 3.3 Splitting text into sentences
- 3.4 Ingestion: fill the database
- 3.5 Vector search
- 3.6 From sentences to context

PHASE 4 **The Full Pipeline and Prompt Engineering**

- 4.1 The retrieval pipeline

4.2 Merging overlapping windows
4.3 The Retriever, complete
4.4 Prompt engineering
4.5 Generating the answer
4.6 Run it
4.7 When answers go wrong

PHASE 5 **Beyond the Course**

5.1 Measure before you improve
5.2 Indexes: HNSW and full-text
5.3 Hybrid search
5.4 Re-ranking with the LLM
5.5 A web chat UI
5.6 Where to go next

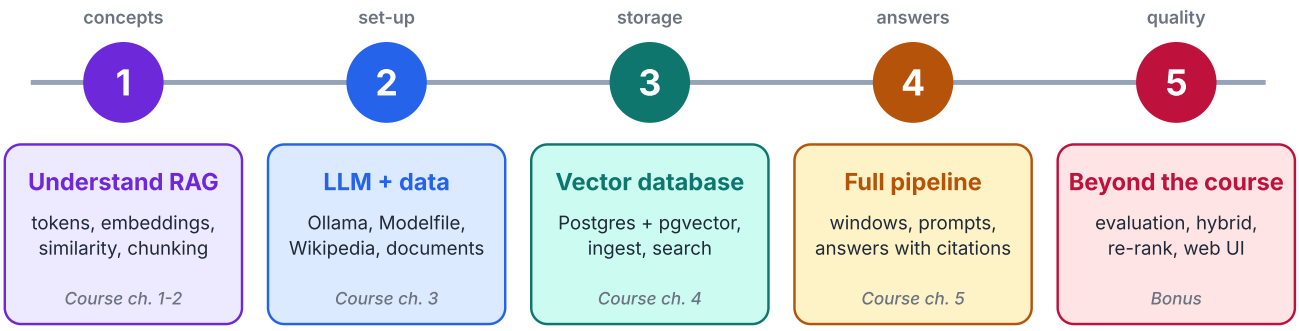
APPENDICES **Reference**

A. Command reference
B. Troubleshooting
C. Complete console app listings
D. Python course to .NET mapping
E. Glossary
F. Licences of everything used

START HERE

The Plan

This book turns a Python video course into a hands-on .NET project. Every chapter of the course becomes one **phase**. In each phase you learn the ideas, write the C# code and run it on your own laptop. By the end you will have a working question-answering system over your own documents. Every line of it will be code you understand, and every component will run locally for free.



The five phases. Phases 1–4 follow the original course chapters. Phase 5 covers topics the course left out.

What you will build

A **RAG** (retrieval-augmented generation) system. You ask a question in plain English. The system finds the relevant passages in a collection of documents, gives them to a local language model, and the model answers using only those passages, citing its sources. You will be able to run it from the console and, in Phase 5, from a small web page.

```
> rag ask "Is the Dominican Republic a member of the United Nations?"

Retrieved 5 context window(s) in 0.6s:
  [Source 1] Human rights in the Dominican Republic (sentences 0-9)
  [Source 2] United Nations (sentences 41-47)
  ...
Answer:
Yes. The Dominican Republic is a member of the United Nations [Source 1].
```

The phases at a glance

Phase	Course chapter	You will learn	You will have at the end	Time
1 Understanding RAG	1–2: Introduction, concepts	Tokens, how LLMs generate text, embeddings, cosine similarity, chunking, vector databases, prompts	A clear mental model of every moving part	2–3 h

Phase	Course chapter	You will learn	You will have at the end	Time
2 LLM and data	3: Preparing your LLM and data	Installing Ollama, Modelfiles, calling models from C#, building a corpus, extracting text from PDF/Word/PowerPoint	Models running locally, a solution that talks to them, ~100 articles on disk	3–4 h
3 Vector database	4: Database and retrieval	pgvector, the EF Core schema, sentence splitting, ingestion, vector search	A searchable database of ~10,000 embedded sentences	2–3 h + ingestion time
4 Full pipeline	5: Packaging, pipeline and prompt engineering	Context windows and overlap merging, prompt design, streaming answers with citations	A working `rag ask` command	3 h
5 Beyond the course	(bonus)	Measuring quality, indexes, hybrid search, re-ranking, a web UI	A system you can measure and improve	3–4 h

How each phase is organised

Every phase follows the same pattern, so you always know where you are:

1. **Concepts first.** The ideas in plain language, with diagrams. No AI background is assumed.
2. **Code.** Every file is printed in full, taken directly from the accompanying solution. If you prefer, unzip `RagFromScratch.zip` and read along instead of typing.
3. **Run it.** The exact commands, and what you should see.
4. **Course comparison.** Purple boxes explain what the Python course did and what we do differently, usually fixing a bug or a gap.
5. **Checkpoint.** A short list to confirm everything works before you move on.

The technology stack

Purpose	Tool	Licence	Replaces (Python course)
Language and runtime	.NET 10, C#	MIT	Python 3
Run models locally	Ollama (on Windows)	MIT	Ollama on Codespaces
Chat model	`granite4.1:3b` (IBM Granite 4.1)	Apache 2.0	DeepSeek-R1 / Gemma 3
Embedding model	`qwen3-embedding:0.6b`	Apache 2.0	SFR-Embedding-Mistral (non-commercial)

Purpose	Tool	Licence	Replaces (Python course)
AI client library	OllamaSharp + Microsoft.Extensions.AI	MIT	`ollama` Python package, sentence-transformers
Database	PostgreSQL 16 + pgvector (Docker)	PostgreSQL	Same
Database access	EF Core + Npgsql + Pgvector.EntityFrameworkCore	MIT / PostgreSQL	SQLAlchemy + psycopg2
Documents	PdfPig, Open XML SDK	Apache 2.0 / MIT	Docling
Corpus	Wikipedia (MediaWiki API)	CC BY-SA content	`wikipedia` package
Containers	Docker Desktop (free for personal use)	(free tier)	GitHub Codespaces

NOTE

Why Granite instead of the course's models? DeepSeek-R1 is a "reasoning" model that writes long `<think>` passages before answering. The course itself drops it for that reason. Gemma 3 is free to download, but its licence is not an open-source licence. IBM's Granite 4.1 uses the standard Apache 2.0 licence, has no thinking mode, and was built for RAG and business tasks. The 3-billion-parameter version runs well on a laptop.

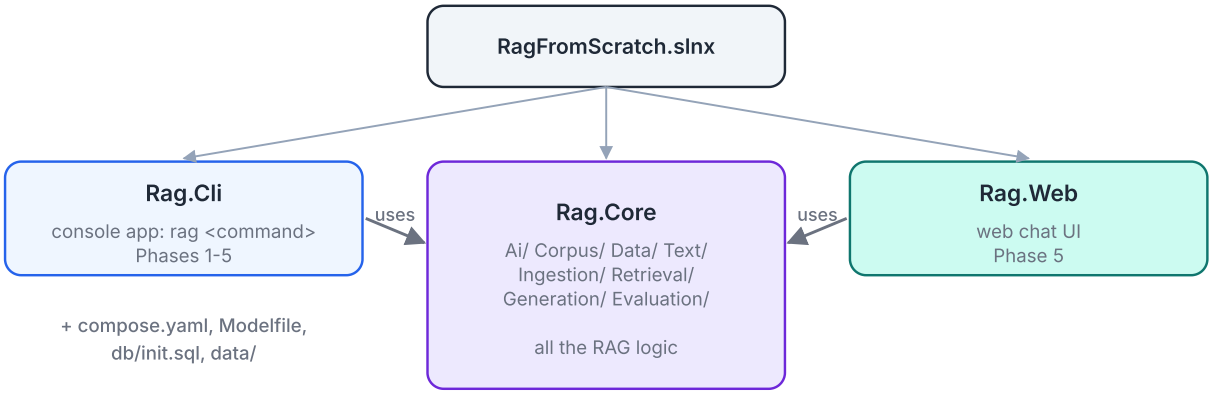
Your hardware

ON YOUR LAPTOP

You have an Intel i7-6700HQ (4 cores, 8 threads), **32 GB RAM**, an NVIDIA GTX 960M (4 GB) and an SSD + HDD. That is enough for this whole book:

- **RAM** is what matters most for local models. 32 GB comfortably holds the 3B chat model, the embedding model, PostgreSQL and Docker at once.
- **CPU-only** generation with a 3B model gives a few to around ten words per second. That's fine for learning.
- **The GTX 960M** can speed things up. Ollama supports it, but it needs NVIDIA driver 570 or newer. If the GPU isn't used, everything still works on the CPU.
- **Use the SSD** for Docker's disk image and Ollama's models. Loading a 2 GB model from a spinning HDD is painfully slow.
- **Start small:** this book uses about 100 Wikipedia articles instead of the course's 487, so ingestion takes minutes instead of hours.

The solution you will build



The solution has three projects. All of the RAG logic lives in Rag.Core. The console app and the web app are thin front ends over it.

The console app has one command per step of the book:

Command	Phase	What it does
`rag selftest`	any	Runs offline checks of the pure logic (no Ollama or database needed)
`rag check`	2	Confirms Ollama, both models and PostgreSQL are reachable
`rag similarity "a" "b"`	2	Embeds sentences and shows how similar their meanings are
`rag corpus`	2	Downloads Wikipedia articles as text files
`rag extract`	2	Converts your PDF, DOCX and PPTX files into text
`rag ingest`	3	Splits, embeds and stores every text file
`rag search "..."`	3	Shows the raw nearest sentences for a query
`rag ask "..."`	4	The full RAG answer, with sources
`rag index`	5	Adds vector and full-text indexes
`rag eval`	5	Measures retrieval quality

You run commands from the solution folder with `dotnet run --project src/Rag.Cli -- <command>`. To save typing, Appendix A shows how to create a short `rag` alias.

NOTE

About the code in this book. Every listing is pulled automatically from the accompanying solution, so the book and the zip file always match. The pure logic (sentence splitting, cosine maths, window merging, rank fusion) was compiled and tested with `rag selftest`. The calls into OllamaSharp, EF Core, pgvector, PdfPig and the Open XML SDK follow each library's documentation. Package versions change, so if `dotnet build` reports an API difference, Appendix B covers the usual fixes.

PHASE 1

Understanding RAG

IN THIS PHASE

Course chapters 1–2 (“Introduction” and “Concepts of RAG models”). No code yet. This phase gives you the mental model behind every line you will write later: what problem RAG solves, how a language model produces text, what tokens and embeddings are, how similarity is measured, why we split documents into sentences, and what goes into a prompt.

1.1 The problem RAG solves

A large language model (LLM) such as Granite, Llama or GPT learned from a huge snapshot of public text. It is excellent at language, but it has three weaknesses for real applications:

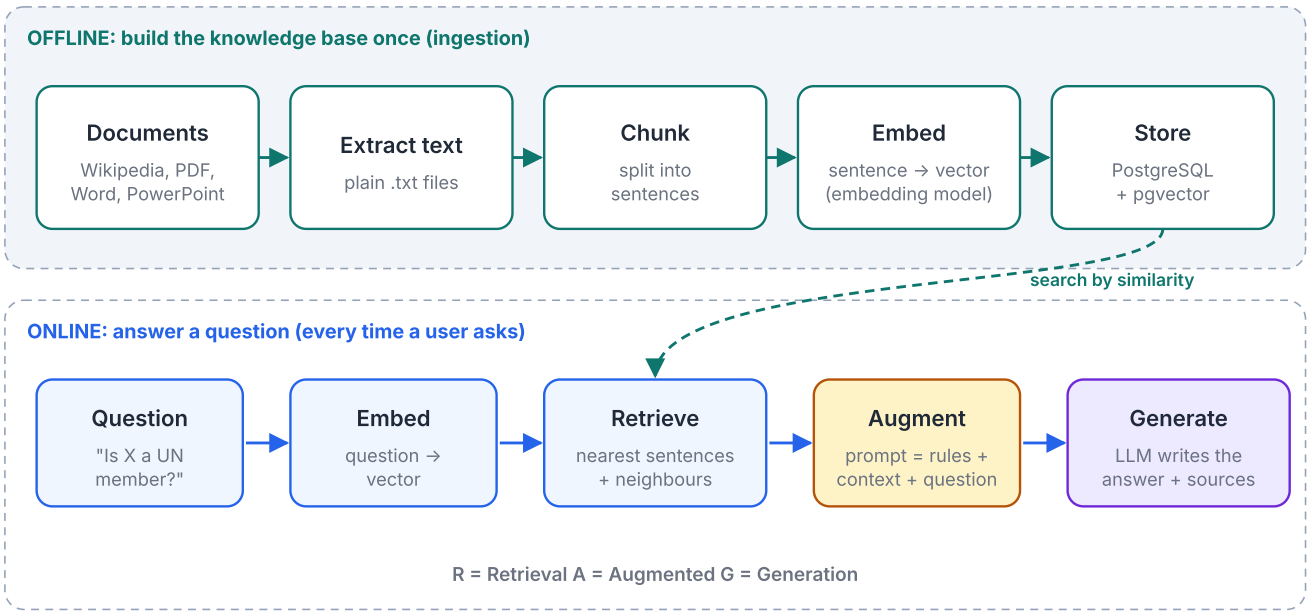
- **It does not know your data.** Your company handbook, your contracts and last week’s report were never in its training data.
- **It can be out of date.** Its knowledge stops at its training cut-off.
- **It can “hallucinate”.** When it doesn’t know, it may still produce a fluent, confident and wrong answer.

The course opens with the question: *how can you tame the wild outputs of LLMs and get them to talk only about the topics you choose?* The answer is **Retrieval-Augmented Generation**. Instead of asking the model to answer from memory, you first **retrieve** the relevant passages from your own documents, **augment** the question with those passages, and then let the model **generate** an answer based on them.

An analogy: an LLM on its own is like a student in a closed-book exam. RAG makes it an open-book exam, where you hand the student exactly the right pages.

1.2 The big picture

A RAG system has two halves. The **offline** half prepares your documents once. The **online** half runs every time someone asks a question.

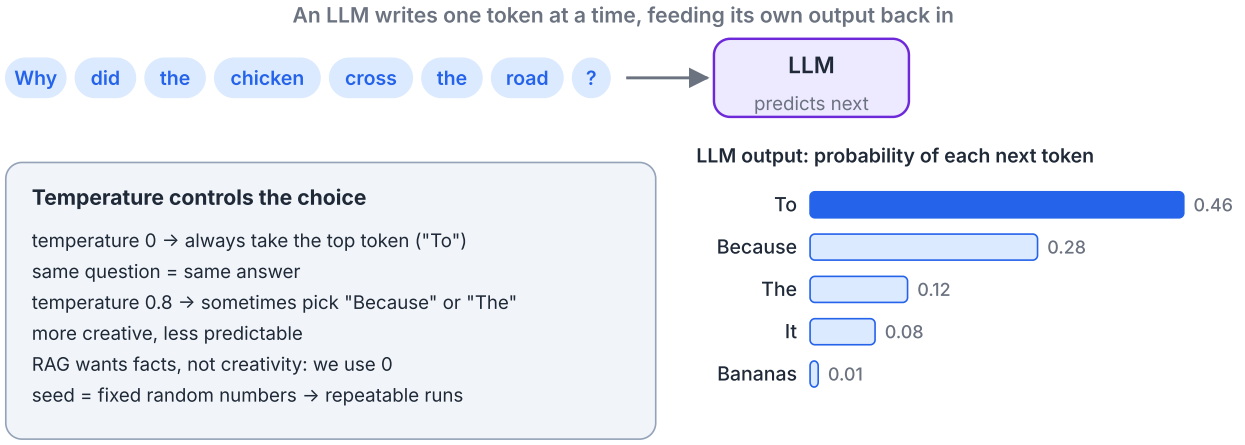


The two halves of RAG. Each box corresponds to code you will write: extract (Phase 2), chunk, embed and store (Phase 3), retrieve, augment and generate (Phase 4).

Keep this picture in mind. Every phase of the book fills in some of its boxes.

1.3 How an LLM writes text

An LLM does one thing over and over: **given the text so far, predict the next token.** It appends that token to the text and repeats. A whole answer is just this loop running a few hundred times.



Next-token prediction and the effect of temperature.

At each step the model produces a probability for every token in its vocabulary. **Temperature** decides how a token is picked from those probabilities:

- **Temperature 0** always takes the most likely token. Ask the same question twice and you get the same answer. That is what we want for RAG, where we care about facts, not creativity.
- **Higher temperatures** sometimes pick less likely tokens. The output becomes more varied and creative, and less predictable.

A **seed** fixes the computer's pseudo-random number generator, so even sampling with randomness becomes repeatable. The course sets temperature 0 and seed 42 in Phase 2, and so will we.

The context window

The model can only "see" a limited amount of text at once: the **context window**, measured in tokens. Everything we send (instructions, retrieved passages, the question) and everything the model writes back must fit inside it. We will set it to 8,192 tokens, which is plenty for five passages of about seven sentences each. When a prompt is too long, Ollama silently cuts text from the **start**. That is why our prompt always puts the question **last**.

Training versus inference

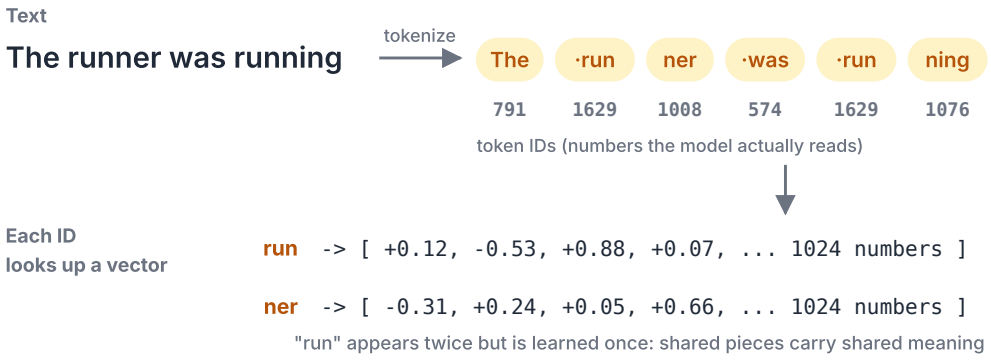
Training adjusts the model's billions of internal numbers (its *weights*) using huge amounts of data and computing power. **Inference** is simply *using* a trained model to generate text. The course points out that over 99% of people working with LLMs only do inference, and that's all we do here. We download a finished model and run it.

Quantization

A model's weights are numbers stored with a certain precision. **Quantization** stores them with fewer bits. For example, 4-bit instead of 16-bit cuts memory use by about four times, with only a small loss of quality. Ollama's models come quantized by default. That is why a 3-billion-parameter model fits in about 2 GB and runs on a laptop.

1.4 Tokens: how text becomes numbers

Computers only process numbers, so text is first split into **tokens**: common words, word pieces and punctuation. Each token has an ID number.



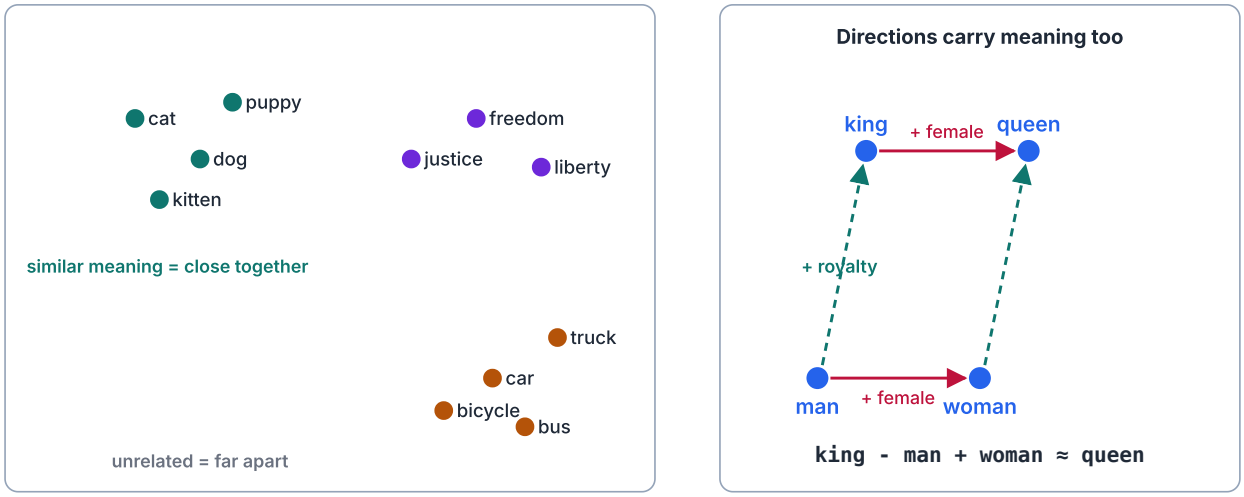
Tokenization splits words into reusable pieces, and each piece maps to a vector.

The course's example is "runner", which becomes "run" + "er". The suffix "-er" appears in thousands of words (runner, teacher, writer), so the model can learn its meaning once and reuse it. Modern models use an algorithm called **byte-pair encoding** to choose the pieces automatically. As a rule of thumb, one token is about three-quarters of an English word.

1.5 Embeddings: meaning as numbers

An **embedding** is a list of numbers (a *vector*) that represents the meaning of a piece of text. Our embedding model, `qwen3-embedding:0.6b`, turns any sentence into a list of exactly **1,024 numbers**.

You can think of those 1,024 numbers as coordinates in a 1,024-dimensional space. The key property is that **texts with similar meanings end up close together**:



A 2-D slice of embedding space (real ones have 1024 axes)

Embedding space: related things cluster together, and directions encode relationships.

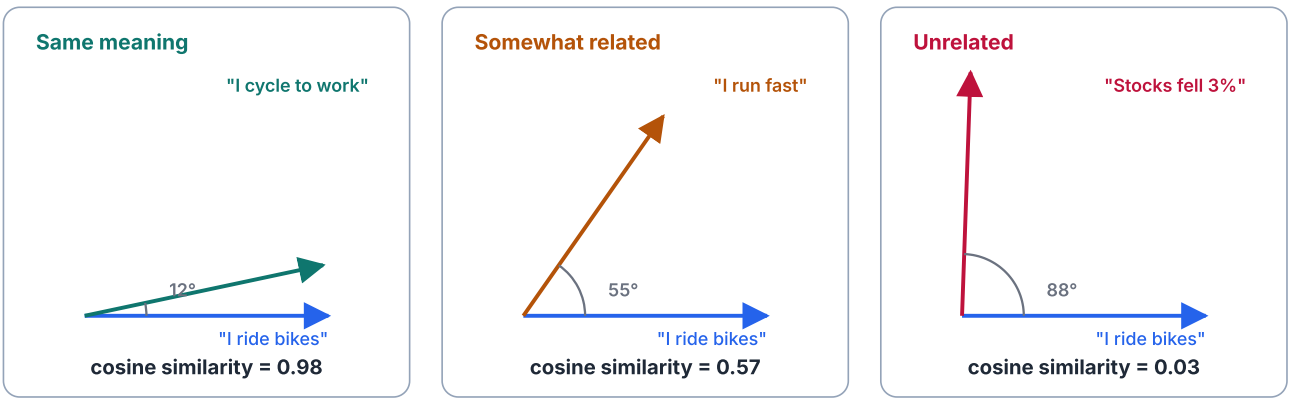
The course's classic example: start at "man", move in the "royalty" direction and you reach "king". Do the same from "woman" and you reach "queen". The model was never told what royalty or gender is. It learned these directions on its own.

How can a model learn meaning just by reading? Through the **distributional hypothesis**: words that appear in similar contexts have similar meanings. "Queen" and "king" appear near words like *throne*, *crowned* and *palace*, so they end up with similar vectors. Early models like Word2Vec (Google, 2013) learned one vector per word. Modern embedding models read the whole sentence and use *attention* to weigh how the words relate to each other, so "bank" in "river bank" and "bank account" get different vectors.

> **NOTE**
>
> **One vector, one idea.** An embedding is a single point in space, so it can only represent one meaning well. A sentence about two unrelated things gets a vector half-way between them, close to neither. That is the reason behind chunking (section 1.7).

1.6 Measuring similarity: cosine

Once texts are vectors, "how similar is the meaning?" becomes "how close are the vectors?". The standard measure is **cosine similarity**, the cosine of the angle between two vectors:



Cosine similarity compares directions, not lengths. Small angle = similar meaning.

- **1.0**: the vectors point the same way (same meaning)
- **about 0**: they are perpendicular (unrelated)
- **−1.0**: they point opposite ways (rare with text embeddings)

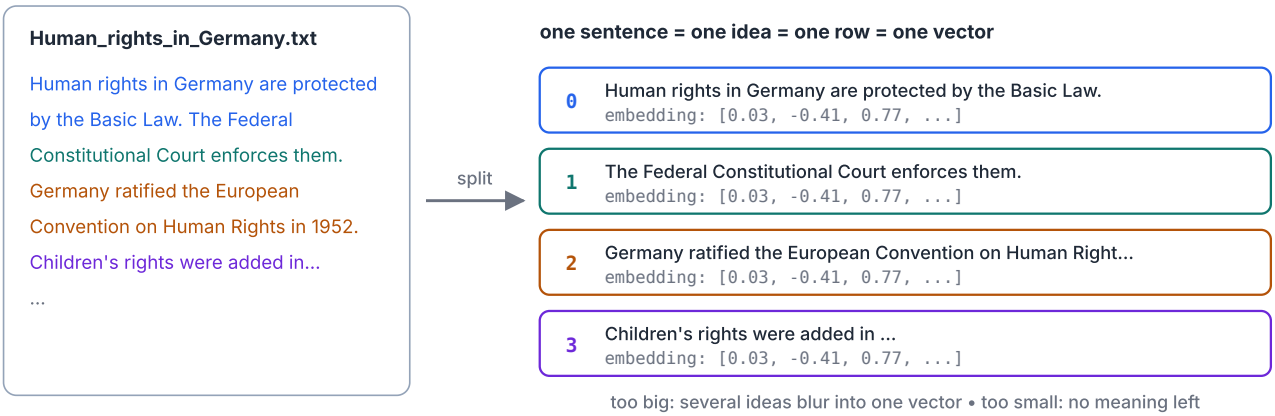
The formula is short. Multiply the vectors element by element and add up the results (the *dot product*), then divide by both lengths:

```
                a1*b1 + a2*b2 + ... + an*bn
cosine(a, b) = ----------------------------------
               sqrt(a1² + ... + an²) * sqrt(b1² + ... + bn²)
```

Databases usually work with **cosine distance**, which is `1 − similarity`: 0 means identical, and smaller is closer. pgvector's `<=>` operator returns exactly that. You will write the formula yourself in C# in Phase 2 (`VectorMath.cs`) and watch PostgreSQL compute it for you in Phase 3.

1.7 Chunking: one idea per vector

Documents are long, and embeddings work best on one idea at a time, so we split documents into **chunks**. The course chooses **one sentence per chunk**: a sentence usually expresses one idea, and the course instructor found that this works well in practice.



Chunking: each sentence becomes a row with its own vector, its file name and its position in the file.

Chunk size is a trade-off. Chunks that are too large blur several ideas into one vector. Chunks that are too small lose meaning (a sentence like "It was ratified in 1952." means little on its own). The course solves the second problem neatly. It searches at the sentence level but, before answering, it adds the **neighbouring sentences** around each match (three before and three after). Storing each sentence's **file name** and **position** is what makes that possible.

1.8 Vector databases

A vector database stores text together with its embeddings, and can quickly answer the question *"which stored vectors are closest to this one?"*. We use **PostgreSQL** with the **pgvector** extension, which adds a `vector` column type and distance operators to the database you may already know. A search is ordinary SQL:

```
SELECT content FROM text_embeddings
ORDER BY embedding <=> '[0.12, -0.53, ...]'   -- cosine distance to the question
LIMIT 5;
```

Without an index, this compares the question with every row: an **exact search**. That is perfectly fast for tens of thousands of rows. Phase 5 adds an **HNSW index** for approximate but much faster search over millions of rows.

1.9 Prompts: putting it together

The last step packages everything for the model. The course splits the prompt into a **static** part and a **dynamic** part:

- **Static** (written once, stored in the Ollama Modelfile): who the model is, the rules ("answer only from the provided context; if the answer isn't there, say so"), and one or two example exchanges (**few-shot prompting**).
- **Dynamic** (built for every question in C#): the retrieved passages, then the question.

Under the hood, chat models are trained with special **role tokens** that mark where system instructions, user messages and assistant replies begin and end, for example `<|system|>`, `<|user|>` or `<|im_end|>`. You never type these yourself. Ollama inserts them using each model's **chat template**. It is still useful to know they exist. Phase 4 shows you how to look at them.

Prompt engineering is, as the course says, as much art as science. Expect your first answers to be imperfect and to improve with iteration.

1.10 Mapping the concepts to the code

Concept	Where it lives in your solution	Phase
LLM, temperature, seed, context window	`Modelfile`	2
Calling the chat and embedding models	`Ai/AiClients.cs`, `Ai/EmbeddingService.cs`	2

Concept	Where it lives in your solution	Phase
Cosine similarity	`Text/VectorMath.cs`	2
Corpus (Wikipedia, documents)	`Corpus/*.cs`	2
Chunking	`Text/SentenceSplitter.cs`	3
Vector database	`Data/*.cs`, `compose.yaml`	3
Ingestion (offline half)	`Ingestion/Ingestor.cs`	3
Retrieval, neighbouring sentences	`Retrieval/Retriever.cs`, `Retrieval/WindowMerger.cs`	3–4
Prompt and generation	`Modelfile`, `Generation/*.cs`	4
Measuring quality, hybrid search	`Evaluation/`, `Retrieval/RankFusion.cs`	5

CHECKPOINT

You are ready for Phase 2 if you can explain, in your own words:

1. Why an LLM alone may answer questions about your documents wrongly, and how RAG helps.
2. What temperature 0 does, and why the question goes at the end of the prompt.
3. What an embedding is, and what a cosine similarity of 0.9 versus 0.1 tells you.
4. Why we embed single sentences but send the model several neighbouring sentences.
5. The difference between the static and dynamic parts of a prompt.

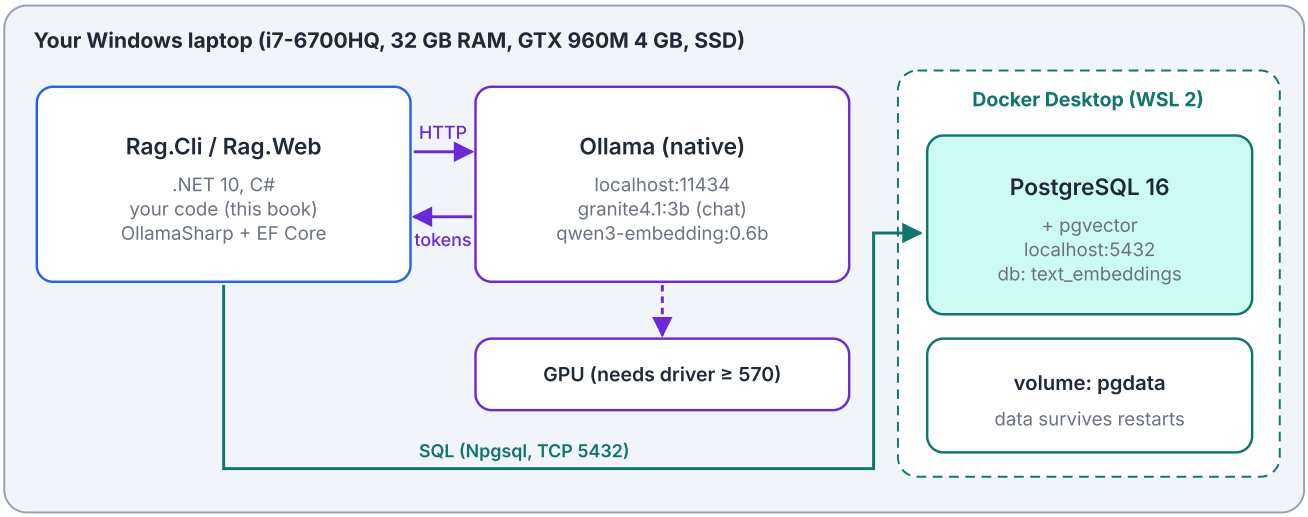
PHASE 2

Setting Up Your LLM and Data

IN THIS PHASE

Course chapter 3 (“Preparing your LLM and data”). You will install every tool, run the models locally, create a custom model with a Modelfile, start PostgreSQL, create the .NET solution, call the models from C#, and collect the documents your RAG system will answer from: Wikipedia articles plus your own PDF, Word and PowerPoint files.

2.1 What runs where



Everything runs on your laptop. Ollama runs natively on Windows for the easiest GPU access. PostgreSQL runs in Docker.

COMPARED WITH THE PYTHON COURSE

The course runs everything inside a Linux GitHub Codespace (a paid cloud machine beyond the free hours), and had to upgrade to the largest machine type for the 8B model. We run locally for free instead. Ollama runs as a normal Windows app, so there is no `ollama serve` terminal to keep open. PostgreSQL comes from a ready-made Docker image with pgvector already installed, which replaces about a dozen `apt` and `pg_hba.conf` steps.

2.2 Install the tools

Open **PowerShell** and install the basics with `winget`, the package manager that ships with Windows:

```
winget install Microsoft.DotNet.SDK.10      # .NET 10 SDK
winget install Git.Git                      # Git
winget install VSCodium.VSCodium            # editor (or use VS Code / Visual Studio)
```

Close and reopen PowerShell, then check: `dotnet --version` should print `10.0.x`.

In VSCodium, install the **DotRush** extension (C# support, MIT licence) from the Extensions panel. If you already use Visual Studio or VS Code with C# Dev Kit for personal projects, those work too.

Graphics driver (optional, for GPU speed)

Ollama can use your GTX 960M, but only with NVIDIA driver **570 or newer**. Install the latest driver NVIDIA offers for the GTX 960M from nvidia.com, then check in PowerShell:

```
nvidia-smi      # the top line shows "Driver Version: 5xx.xx"
```

If the GPU can't be used, nothing breaks. Ollama falls back to the CPU.

Docker Desktop settings

You already have Docker Desktop. Two settings make a big difference on your laptop:

1. **Put the disk image on the SSD.** Settings → Resources → Advanced → *Disk image location* → choose a folder on the SSD (the CT1000MX500).
2. **Limit WSL 2's memory.** Create the file `C:\Users\<you>\.wslconfig` with:

```
[wsl2]
memory=8GB
processors=4
```

Run `wsl --shutdown` and restart Docker Desktop. Ollama runs outside WSL, so 8 GB is plenty for PostgreSQL, and the rest of your 32 GB stays free for the models.

2.3 Install Ollama and the models

Download **OllamaSetup.exe** from ollama.com/download and run it. Ollama starts automatically, shows a llama icon in the system tray, and listens on `http://localhost:11434`.

> **ON YOUR LAPTOP**
>
> Ollama stores models in `C:\Users\<you>\.ollama\models` by default. If your C: drive is the **HDD**, move models to the SSD first. Add a user environment variable `OLLAMA_MODELS` = `D:\ollama\models` (use your SSD's drive letter), then quit and restart Ollama from the tray icon.

Pull the two models (about 2.6 GB in total):

```
ollama pull granite4.1:3b           # chat model: answers questions (Apache 2.0)
ollama pull qwen3-embedding:0.6b    # embedding model: text -> 1024 numbers (Apache 2.0)
```

Try the chat model interactively:

```
ollama run granite4.1:3b
>>> Why did the chicken cross the road?
...
>>> /bye
```

While a model is loaded, `ollama ps` shows whether it runs on the GPU. The PROCESSOR column says `100% GPU`, a CPU/GPU split, or `100% CPU`.

COMPARED WITH THE PYTHON COURSE

The course used `deepseek-r1:8b`, then `qwen3:0.6b` to save money, and finally `gemma3:12b`. DeepSeek-R1 is a reasoning model whose long `<think>` output slowed every answer. The course says itself that it is not suitable for RAG. A 12B model would be very slow on your laptop's CPU. `granite4.1:3b` gives you an open licence, no thinking overhead and a laptop-friendly size. If answers feel too simple later, try `granite4.1:8b`: it is the same family, only bigger and slower.

2.4 The Modelfile: a custom model

An Ollama **Modelfile** is like a Dockerfile for models. It starts from a base model and adds settings and instructions, and `ollama create` turns it into a new named model. Here is the complete Modelfile for this book. In this phase we only care about the first part (`FROM` and `PARAMETER`). The `SYSTEM` and `MESSAGE` lines are the prompt engineering of Phase 4.

Modelfile

```
# Custom RAG chat model.  Build it with:   ollama create rag-granite -f Modelfile
# Inspect the base model's own settings:  ollama show --modelfile granite4.1:3b

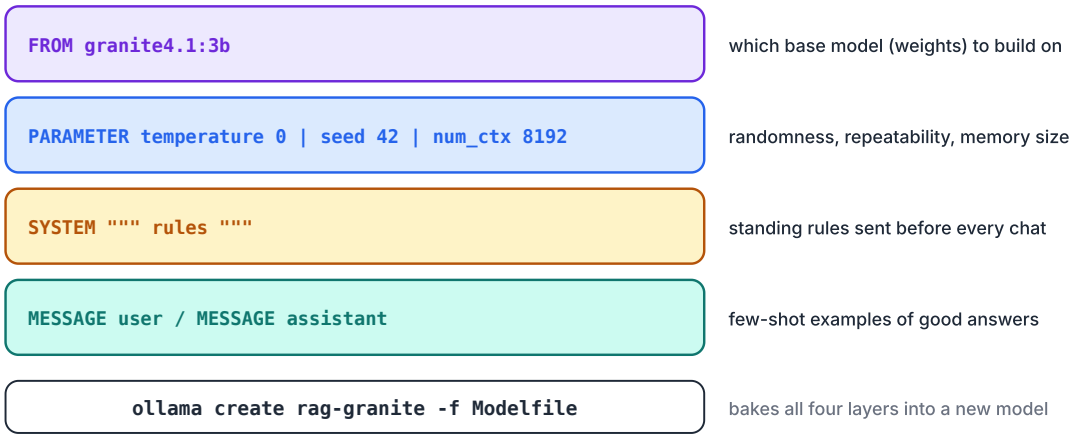
FROM granite4.1:3b

# Deterministic answers: always pick the most likely next token.
PARAMETER temperature 0
# Fixed random seed, so the same question gives the same answer.
PARAMETER seed 42
# Context window in tokens. 5 windows x 7 sentences + question fits comfortably in 8192.
PARAMETER num_ctx 8192

SYSTEM """You are a careful assistant that answers questions using ONLY the content provided between
<context> and </context>.

Rules:
1. Use only facts stated in the context. Do not use outside knowledge.
2. After each fact, cite its source label, for example [Source 2].
3. If the context does not contain the answer, reply exactly: Given the information provided, I'm
unable to answer your question.
4. Keep answers short and direct: a few sentences unless the question asks for detail."""

# Few-shot examples: show the model what a good answer and a refusal look like.
MESSAGE user <context>[Source 1: Eiffel Tower] The Eiffel Tower is a wrought-iron tower in Paris,
France. It was completed in 1889.</context> Question: When was the Eiffel Tower completed?
MESSAGE assistant The Eiffel Tower was completed in 1889 [Source 1].
MESSAGE user <context>[Source 1: Eiffel Tower] The Eiffel Tower is a wrought-iron tower in Paris,
France. It was completed in 1889.</context> Question: How tall is Mount Everest?
MESSAGE assistant Given the information provided, I'm unable to answer your question.
```



The four layers of a Modelfile.

Line	Meaning
`FROM granite4.1:3b`	Start from these model weights
`PARAMETER temperature 0`	Always pick the most likely next token, giving deterministic, factual answers
`PARAMETER seed 42`	Fix the random seed so runs are repeatable
`PARAMETER num_ctx 8192`	Context window of 8,192 tokens (the course used 4,096; we need room for five passages)
`SYSTEM """..."""`	Standing instructions sent before every conversation (Phase 4)
`MESSAGE user / assistant`	Example exchanges that show the model the expected answer style (Phase 4)

Save the file as `Modelfile` (no extension) in the solution folder and create the model:

```
ollama create rag-granite -f Modelfile
ollama run rag-granite "Reply with exactly: OK"
```

2.5 Start PostgreSQL with pgvector

Two small files describe the database. `compose.yaml` tells Docker which image to run, with what password, on which port, and where to keep the data:

compose.yaml

```
# PostgreSQL 16 with the pgvector extension pre-installed.
# Start:   docker compose up -d
# Stop:    docker compose down          (your data survives in the "pgdata" volume)
# Wipe:    docker compose down -v       (deletes the database too)
#
# Ollama runs natively on Windows (better GPU support on laptops), so it is not in this file.
# If you prefer Ollama in Docker as well, uncomment the "ollama" service below.

services:
  postgres:
```

```
    image: pgvector/pgvector:pg16
    container_name: rag-postgres
    restart: unless-stopped
    ports:
      - "5432:5432"
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: text_embeddings
    volumes:
      - pgdata:/var/lib/postgresql/data
      # Runs once, when the database is first created: enables the vector extension.
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro

  # ollama:
  #   image: ollama/ollama
  #   container_name: rag-ollama
  #   ports:
  #     - "11434:11434"
  #   volumes:
  #     - ollama:/root/.ollama

volumes:
  pgdata:
  # ollama:
```

`db/init.sql` runs once, the first time the database is created. It switches on the pgvector extension:

db/init.sql

```
-- Executed automatically the first time the Postgres container creates its database.
-- Creating the extension up-front means the .NET app already knows the "vector" type
-- the very first time it connects.
CREATE EXTENSION IF NOT EXISTS vector;
```

Start it, then confirm the extension is installed:

```
docker compose up -d
docker exec -it rag-postgres psql -U postgres -d text_embeddings -c "\dx"
#   Name   | Version | ...
#  vector  | 0.8.x   | ...
```

COMPARED WITH THE PYTHON COURSE

The course installs PostgreSQL 16 and pgvector with `apt`, switches `pg_hba.conf` from `peer` to `md5`, sets a password with `ALTER USER`, creates the database and runs `CREATE EXTENSION vector` by hand. It also had to restart the service each time the Codespace restarted. The `pgvector/pgvector:pg16` image does all of that. `restart: unless-stopped` brings it back with Docker Desktop, and the `pgdata` volume keeps your data.

2.6 Create the .NET solution

You can unzip `RagFromScratch.zip` and skip to section 2.7, or create the structure yourself:

```
mkdir RagFromScratch; cd RagFromScratch
dotnet new sln -n RagFromScratch
dotnet new classlib -n Rag.Core -o src/Rag.Core
```

```
dotnet new console  -n Rag.Cli  -o src/Rag.Cli
dotnet new web      -n Rag.Web  -o src/Rag.Web
dotnet sln add src/Rag.Core src/Rag.Cli src/Rag.Web
dotnet add src/Rag.Cli reference src/Rag.Core
dotnet add src/Rag.Web reference src/Rag.Core

dotnet add src/Rag.Core package OllamaSharp
dotnet add src/Rag.Core package Npgsql.EntityFrameworkCore.PostgreSQL
dotnet add src/Rag.Core package Pgvector.EntityFrameworkCore
dotnet add src/Rag.Core package UglyToad.PdfPig
dotnet add src/Rag.Core package DocumentFormat.OpenXml
```

Delete the template's `Class1.cs`. The project files should end up like these:

src/Rag.Core/Rag.Core.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <RootNamespace>Rag.Core</RootNamespace>
  </PropertyGroup>

  <ItemGroup>
    <!-- Talks to Ollama; also brings Microsoft.Extensions.AI.Abstractions (IChatClient,
IEmbeddingGenerator) -->
    <PackageReference Include="OllamaSharp" Version="5.5.2" />
    <!-- EF Core provider for PostgreSQL -->
    <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.0" />
    <!-- pgvector support for EF Core: the Vector type and CosineDistance() -->
    <PackageReference Include="Pgvector.EntityFrameworkCore" Version="0.3.0" />
    <!-- Document readers (the .NET replacement for Docling) -->
    <PackageReference Include="UglyToad.PdfPig" Version="0.1.10" />
    <PackageReference Include="DocumentFormat.OpenXml" Version="3.3.0" />
  </ItemGroup>

</Project>
```

src/Rag.Cli/Rag.Cli.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <OutputType>Exe</OutputType>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <AssemblyName>rag</AssemblyName>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\Rag.Core\Rag.Core.csproj" />
  </ItemGroup>

</Project>
```

NuGet package	Why we need it
OllamaSharp	Talks to Ollama. It implements Microsoft's standard `IChatClient` and `IEmbeddingGenerator` interfaces (it brings in `Microsoft.Extensions.AI.Abstractions`).
Npgsql.EntityFrameworkCore.PostgreSQL	EF Core provider for PostgreSQL
Pgvector.EntityFrameworkCore	The `Vector` type and `CosineDistance()` for EF Core
UglyToad.PdfPig	Reads text from PDF files
DocumentFormat.OpenXml	Reads Word (.docx) and PowerPoint (.pptx) files

COMPARED WITH THE PYTHON COURSE

The Python course creates a virtual environment (`python3 -m venv rag_env`) and installs packages with pip. In .NET, the `.csproj` file plays the role of both the venv and `requirements.txt` : packages are restored per project, automatically, on `dotnet build` .

2.7 Settings in one place

Every tunable value lives in one record. Each value has a sensible default and can be overridden with an environment variable, so trying another model never needs a code change.

src/Rag.Core/RagSettings.cs

```
namespace Rag.Core;

/// <summary>
/// All the knobs of the RAG system in one place.
/// Every value can be overridden with an environment variable, so you never
/// have to edit code to try a different model or database.
/// </summary>
public sealed record RagSettings
{
    // ---- Ollama -------------------------------------------------------------
    public string OllamaUrl { get; init; } = Env("RAG_OLLAMA_URL", "http://localhost:11434");

    /// <summary>The custom chat model we create from the Modelfile (Phase 2 / Phase 4).</summary>
    public string ChatModel { get; init; } = Env("RAG_CHAT_MODEL", "rag-granite");

    /// <summary>The plain base model, used where our RAG system prompt would get in the way (e.g.
re-ranking).</summary>
    public string BaseChatModel { get; init; } = Env("RAG_BASE_MODEL", "granite4.1:3b");

    public string EmbeddingModel { get; init; } = Env("RAG_EMBED_MODEL", "qwen3-embedding:0.6b");

    /// <summary>Must match the embedding model: qwen3-embedding:0.6b = 1024, nomic-embed-text =
768.</summary>
    public int EmbeddingDimensions { get; init; } = int.Parse(Env("RAG_EMBED_DIMS", "1024"));

    // ---- PostgreSQL ---------------------------------------------------------
    public string ConnectionString { get; init; } = Env("RAG_PG",
        "Host=localhost;Port=5432;Database=text_embeddings;Username=postgres;Password=postgres");

    // ---- Files --------------------------------------------------------------
    /// <summary>Folder of .txt files that get ingested (Wikipedia articles + extracted documents).
</summary>
```

```
    public string CorpusFolder { get; init; } = Env("RAG_CORPUS", Path.Combine("data",
"all_articles"));

    // ---- Retrieval ---------------------------------------------------------
    /// <summary>Sentences taken before AND after each matching sentence (the course used 3).
</summary>
    public int WindowSize { get; init; } = 3;

    /// <summary>How many distinct context windows we hand to the LLM (the course used 5).</summary>
    public int MaxWindows { get; init; } = 5;

    /// <summary>How many raw matches we fetch before merging overlaps (deliberately larger than
MaxWindows).</summary>
    public int CandidatePool { get; init; } = 50;

    /// <summary>Embedding batch size during ingestion. Lower it if Ollama runs out of memory.
</summary>
    public int EmbeddingBatchSize { get; init; } = 32;

    private static string Env(string name, string fallback) =>
        Environment.GetEnvironmentVariable(name) is { Length: > 0 } value ? value : fallback;
}
```

TIP

To try a different embedding model later, for example `nomic-embed-text` (768 dimensions), set two variables: `$env:RAG_EMBED_MODEL="nomic-embed-text"` and `$env:RAG_EMBED_DIMS="768"`. Then re-ingest with `--reset`, because vectors from different models can't be mixed.

2.8 Talking to the models from C

Microsoft.Extensions.AI defines two standard interfaces, and OllamaSharp's `OllamaApiClient` implements both:

- `IChatClient`: send messages, get a reply (whole or streamed token by token)
- `IEmbeddingGenerator<string, Embedding<float>>`: send text, get vectors

The rest of our code only depends on these interfaces. You could swap Ollama for another provider later without touching the RAG logic.

src/Rag.Core/Ai/AiClients.cs

```
using Microsoft.Extensions.AI;
using OllamaSharp;

namespace Rag.Core.Ai;

/// <summary>
/// Creates the two AI "clients" the whole system needs.
///
/// OllamaApiClient implements Microsoft.Extensions.AI's standard interfaces:
///   IChatClient                                -> talk to an LLM
///   IEmbeddingGenerator&lt;string, Embedding&lt;float&gt;&gt; -> turn text into vectors
/// Because the rest of the code only sees these interfaces, you could later swap Ollama
/// for another provider without touching the RAG logic.
/// </summary>
public static class AiClients
{
    // Local models on a laptop CPU can be slow: allow up to 10 minutes per request.
```

```
    private static HttpClient Http(RagSettings s) =>
        new() { BaseAddress = new Uri(s.OllamaUrl), Timeout = TimeSpan.FromMinutes(10) };

    public static IChatClient CreateChatClient(RagSettings s, string? model = null) =>
        new OllamaApiClient(Http(s), model ?? s.ChatModel);

    public static IEmbeddingGenerator<string, Embedding<float>> CreateEmbeddingGenerator(RagSettings
s) =>
        new OllamaApiClient(Http(s), s.EmbeddingModel);
}
```

The embedding service wraps the generator and adds two things: batching, and the **query instruction**. Qwen3-Embedding is *instruction-aware*: questions should start with a one-line task description, and documents should not. This asymmetry measurably improves search quality. The Python course never mentions it.

src/Rag.Core/Ai/EmbeddingService.cs

```
using Microsoft.Extensions.AI;

namespace Rag.Core.Ai;

/// <summary>
/// Turns text into embedding vectors.
///
/// Qwen3-Embedding is "instruction aware": QUESTIONS should be prefixed with a short task
/// description, DOCUMENTS should not. Getting this asymmetry right noticeably improves
/// retrieval, and the original course never mentions it.
/// </summary>
public sealed class EmbeddingService(IEmbeddingGenerator<string, Embedding<float>> generator)
{
    private const string QueryInstruction =
        "Instruct: Given a question, retrieve passages that answer the question\nQuery: ";

    /// <summary>Embed a user question (with the instruction prefix).</summary>
    public async Task<float[]> EmbedQueryAsync(string question, CancellationToken ct = default)
    {
        var result = await generator.GenerateAsync([QueryInstruction + question], cancellationToken:
ct);
        return result[0].Vector.ToArray();
    }

    /// <summary>Embed document sentences (no prefix), in batches to keep memory use predictable.
</summary>
    public async Task<List<float[]>> EmbedDocumentsAsync(
        IReadOnlyList<string> texts, int batchSize = 32, CancellationToken ct = default)
    {
        var vectors = new List<float[]>(texts.Count);
        for (int i = 0; i < texts.Count; i += batchSize)
        {
            var batch = texts.Skip(i).Take(batchSize).ToList();
            var result = await generator.GenerateAsync(batch, cancellationToken: ct);
            vectors.AddRange(result.Select(e => e.Vector.ToArray()));
        }
        return vectors;
    }

    /// <summary>Embed text as-is (used by the Phase 1 similarity demo).</summary>
    public async Task<float[]> EmbedRawAsync(string text, CancellationToken ct = default)
    {
        var result = await generator.GenerateAsync([text], cancellationToken: ct);
        return result[0].Vector.ToArray();
```

```
    }
}
```

2.9 Cosine similarity by hand

Before handing similarity search over to the database, write the formula from Phase 1 yourself. It's twelve lines:

src/Rag.Core/Text/VectorMath.cs

```
namespace Rag.Core.Text;

/// <summary>
/// The maths behind similarity search, written out by hand so you can see it.
/// (In production pgvector does this inside the database.)
/// </summary>
public static class VectorMath
{
    /// <summary>
    /// Cosine similarity = cos(angle between a and b).
    ///  1.0 = same direction (same meaning), 0 = unrelated, -1 = opposite.
    /// </summary>
    public static double CosineSimilarity(ReadOnlySpan<float> a, ReadOnlySpan<float> b)
    {
        if (a.Length != b.Length) throw new ArgumentException("Vectors must have the same length.");
        double dot = 0, normA = 0, normB = 0;
        for (int i = 0; i < a.Length; i++)
        {
            dot += a[i] * b[i];
            normA += a[i] * a[i];
            normB += b[i] * b[i];
        }
        if (normA == 0 || normB == 0) return 0;
        return dot / (Math.Sqrt(normA) * Math.Sqrt(normB));
    }

    /// <summary>pgvector's "&lt;=&gt;" operator returns cosine DISTANCE = 1 - similarity (smaller =
closer).</summary>
    public static double CosineDistance(ReadOnlySpan<float> a, ReadOnlySpan<float> b) => 1 -
CosineSimilarity(a, b);
}
```

2.10 The console app

`Program.cs` is a small command dispatcher: `rag <command> [arguments]` calls one local function per command. The complete file is in Appendix C (and in the zip). Copy it into `src/Rag.Cli/Program.cs`, along with `SelfTest.cs`, which is also in Appendix C. Each phase walks through the functions it uses. For now, two of them:

src/Rag.Cli/Program.cs: CheckAsync()

```
async Task CheckAsync()
{
    Console.WriteLine($"Ollama:     {settings.OllamaUrl}");
    var chat = AiClients.CreateChatClient(settings);
    var sw = Stopwatch.StartNew();
    var reply = await chat.GetResponseAsync("Reply with exactly: OK", cancellationToken: ct);
    Console.WriteLine($"  chat model  '{settings.ChatModel}' replied '{reply.Text.Trim()}' in
```

```
{sw.Elapsed.TotalSeconds:F1}s");

    var embeddings = new EmbeddingService(AiClients.CreateEmbeddingGenerator(settings));
    var vector = await embeddings.EmbedRawAsync("hello world", ct);
    Console.WriteLine($"  embed model '{settings.EmbeddingModel}' -> {vector.Length} dimensions" +
                      (vector.Length == settings.EmbeddingDimensions ? "" : $"  (!) settings expect
{settings.EmbeddingDimensions}"));

    await using var db = new RagDbContext(settings);
    Console.WriteLine($"PostgreSQL: {(await db.Database.CanConnectAsync(ct) ? "connected" : "NOT
reachable")}");
}
```

src/Rag.Cli/Program.cs: SimilarityAsync()

```
async Task SimilarityAsync(string[] sentences)
{
    if (sentences.Length < 2) { Console.WriteLine("Give at least two sentences in quotes."); return;
}
    var embeddings = new EmbeddingService(AiClients.CreateEmbeddingGenerator(settings));
    var vectors = new List<float[]>();
    foreach (var s in sentences) vectors.Add(await embeddings.EmbedRawAsync(s, ct));

    Console.WriteLine($"Each sentence became a vector of {vectors[0].Length} numbers. First 5 of #1:
" +
                      string.Join(", ", vectors[0].Take(5).Select(v => v.ToString("F3"))) + ",
...\n");
    Console.WriteLine("Cosine similarity (1.00 = same meaning, ~0 = unrelated):");
    for (int i = 0; i < sentences.Length; i++)
        for (int j = i + 1; j < sentences.Length; j++)
            Console.WriteLine($"  {VectorMath.CosineSimilarity(vectors[i], vectors[j]):F2}  \"
{sentences[i]}\"  vs  \"{sentences[j]}\"");
}
```

Run it

```
dotnet build
dotnet run --project src/Rag.Cli -- selftest
dotnet run --project src/Rag.Cli -- check
```

`selftest` runs offline checks of the pure logic: it should end with `All checks passed.`. `check` talks to everything:

```
Ollama:     http://localhost:11434
  chat model  'rag-granite' replied 'OK' in 4.2s
  embed model 'qwen3-embedding:0.6b' -> 1024 dimensions
PostgreSQL: connected
```

Now see Phase 1's ideas in action:

```
dotnet run --project src/Rag.Cli -- similarity "I ride bikes" "I cycle to work" "I run fast" "Stock
prices fell 3% today"
```

```
Each sentence became a vector of 1024 numbers. First 5 of #1: 0.021, -0.043, ...

Cosine similarity (1.00 = same meaning, ~0 = unrelated):
  0.8x  "I ride bikes"  vs  "I cycle to work"
  0.6x  "I ride bikes"  vs  "I run fast"
```

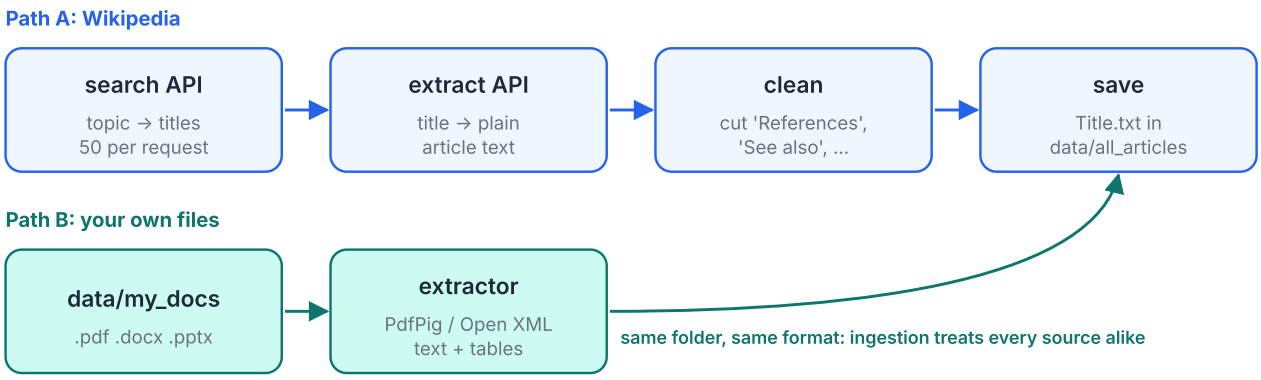
```
  0.2x  "I ride bikes"  vs  "Stock prices fell 3% today"
  ...
```

Your exact numbers will differ, but the order should match: near-synonyms score highest and unrelated sentences lowest. Real embedding models rarely give exactly 0 for unrelated text. What matters is the **ranking**.

> **TIP**
>
> The first call to a model is slow, because Ollama loads it into memory. Later calls are much faster. Ollama unloads a model after 5 idle minutes.

2.11 Build the corpus from Wikipedia

The **corpus** is the collection of documents your RAG system knows about. Like the course, we start with Wikipedia articles on "human rights": a topic with many interlinked articles, which makes for interesting questions.



Two ways into the corpus: Wikipedia articles and your own documents end up as plain .txt files in the same folder.

Instead of a Python package we call the free **MediaWiki API** directly with `HttpClient`. It needs two requests: a *search* that returns titles (50 per page), and an *extract* that returns an article's plain text. Real-world text is messy, so the generator also cuts off the reference and link sections at the end of each article, which would only add noise.

src/Rag.Core/Corpus/WikipediaCorpusGenerator.cs

```
using System.Text;
using System.Text.Json;
using System.Text.RegularExpressions;

namespace Rag.Core.Corpus;

/// <summary>
/// Downloads Wikipedia articles as plain text: the C# version of the course's generate_corpus.py.
/// Uses the free MediaWiki API directly with HttpClient (no extra package needed).
/// </summary>
public sealed partial class WikipediaCorpusGenerator : IDisposable
{
    private const string Api = "https://en.wikipedia.org/w/api.php";

    // Sections that are lists of links/citations, not knowledge. We cut the article there.
```

```
    private static readonly string[] JunkSections =
        ["See also", "References", "Notes", "Further reading", "External links", "Bibliography",
"Sources", "Citations"];

    private readonly HttpClient _http = new();

    public WikipediaCorpusGenerator()
    {
        // Wikimedia asks every client to identify itself with a descriptive User-Agent.
        _http.DefaultRequestHeaders.UserAgent.ParseAdd("RagFromScratchTutorial/1.0 (learning
project)");
    }

    /// <summary>Search Wikipedia for <paramref name="topic"/> and save up to <paramref
name="maxArticles"/> articles as .txt files.</summary>
    public async Task<int> GenerateAsync(string topic, int maxArticles, string outputFolder,
CancellationToken ct = default)
    {
        Directory.CreateDirectory(outputFolder);

        var titles = await SearchTitlesAsync(topic, maxArticles, ct);
        Console.WriteLine($"Found {titles.Count} matching titles for '{topic}'.");

        int saved = 0;
        foreach (var title in titles)
        {
            try
            {
                var text = await GetPlainTextAsync(title, ct);
                if (text.Length < 500) { Console.WriteLine($"  skip (too short): {title}");
continue; }

                var path = Path.Combine(outputFolder, SanitizeFileName(title) + ".txt");
                await File.WriteAllTextAsync(path, text, ct);
                saved++;
                Console.WriteLine($"  [{saved,3}] {title}");
            }
            catch (Exception ex) when (ex is not OperationCanceledException)
            {
                // One bad article must not stop the whole run (same idea as the course's
try/except).
                Console.WriteLine($"  Error processing '{title}': {ex.Message}");
            }

            await Task.Delay(100, ct);   // be polite to Wikipedia's servers
        }

        Console.WriteLine($"Done. Saved {saved} articles to '{outputFolder}'.");
        return saved;
    }

    /// <summary>Pages through the search API (max 50 results per request).</summary>
    private async Task<List<string>> SearchTitlesAsync(string topic, int max, CancellationToken ct)
    {
        var titles = new List<string>();
        int offset = 0;
        while (titles.Count < max)
        {
            int limit = Math.Min(50, max - titles.Count);
            var url = $"{Api}?action=query&list=search&format=json&formatversion=2" +
                      $"&srsearch={Uri.EscapeDataString(topic)}&srlimit={limit}&sroffset=
{offset}&srnamespace=0";

            using var doc = JsonDocument.Parse(await _http.GetStringAsync(url, ct));
            var results = doc.RootElement.GetProperty("query").GetProperty("search");
            foreach (var r in results.EnumerateArray())
titles.Add(r.GetProperty("title").GetString()!);
```

```
            // "continue.sroffset" tells us where the next page starts; no "continue" = no more
results.
            if (!doc.RootElement.TryGetProperty("continue", out var cont)) break;
            offset = cont.GetProperty("sroffset").GetInt32();
        }
        return titles;
    }

    /// <summary>Fetches one article as plain text and removes the reference/link sections.
</summary>
    private async Task<string> GetPlainTextAsync(string title, CancellationToken ct)
    {
        var url = $"{Api}?action=query&prop=extracts&explaintext=1&exsectionformat=wiki&redirects=1"
+
                  $"&format=json&formatversion=2&titles={Uri.EscapeDataString(title)}";

        using var doc = JsonDocument.Parse(await _http.GetStringAsync(url, ct));
        var page = doc.RootElement.GetProperty("query").GetProperty("pages")[0];
        var extract = page.TryGetProperty("extract", out var e) ? e.GetString() ?? "" : "";
        return Clean(extract);
    }

    /// <summary>Real-world data is never clean: cut junk sections and tidy whitespace.</summary>
    internal static string Clean(string text)
    {
        // Headings arrive as "== References ==" (any number of '=').
        foreach (Match m in Heading().Matches(text))
        {
            if (JunkSections.Contains(m.Groups[1].Value.Trim(), StringComparer.OrdinalIgnoreCase))
            {
                text = text[..m.Index];
                break;
            }
        }
        return ExtraBlankLines().Replace(text, "\n\n").Trim();
    }

    /// <summary>Replace anything that is not a letter or digit with '_' (the course's
sanitize_filename).</summary>
    public static string SanitizeFileName(string name)
    {
        var sb = new StringBuilder(name.Length);
        foreach (var c in name) sb.Append(char.IsLetterOrDigit(c) ? c : '_');
        return sb.ToString();
    }

    public void Dispose() => _http.Dispose();

    [GeneratedRegex(@"^=+\s*(.+?)\s*=+\s*$", RegexOptions.Multiline)]
    private static partial Regex Heading();

    [GeneratedRegex(@"\n{3,}")]
    private static partial Regex ExtraBlankLines();
}
```

src/Rag.Cli/Program.cs: CorpusAsync()

```
async Task CorpusAsync()
{
    var topic = Option("--topic") ?? "human rights";
    var max = int.Parse(Option("--max") ?? "100");
    using var generator = new WikipediaCorpusGenerator();
    await generator.GenerateAsync(topic, max, settings.CorpusFolder, ct);
}
```

Run it:

```
dotnet run --project src/Rag.Cli -- corpus --topic "human rights" --max 100
```

```
Found 100 matching titles for 'human rights'.
  [  1] Human rights
  [  2] Universal Declaration of Human Rights
  ...
Done. Saved 9x articles to 'data\all_articles'.
```

A few articles are skipped as too short (disambiguation pages, for example). Open one of the `.txt` files: it should be clean prose with section headings like `== History ==`, and no reference lists.

COMPARED WITH THE PYTHON COURSE

The course asked for 1,000 articles and saved 487. We default to 100 so that ingestion in Phase 3 takes minutes, not hours, on your laptop. Once everything works you can run `corpus` again with a larger `--max` (or another `--topic`). The ingestor only processes new files.

2.12 Your own documents: PDF, Word, PowerPoint

In real projects your corpus is your own files. The course uses IBM's Docling for this, but then never connects its output to the ingestion step. We use two open-source .NET libraries and write the result as `.txt` into the same corpus folder, so ingestion handles every source the same way:

- **PdfPig** reads PDFs. We join the words it detects on each page, which is more reliable than its raw page text.
- **Open XML SDK** reads `.docx` and `.pptx`. For Word, paragraphs and tables are read in document order, and each table row becomes `cell | cell | cell`. For PowerPoint, the text of each slide is read in slide order.

src/Rag.Core/Corpus/DocumentTextExtractor.cs

```
using System.Text;
using DocumentFormat.OpenXml.Packaging;
using UglyToad.PdfPig;
using Drawing = DocumentFormat.OpenXml.Drawing;
using Word = DocumentFormat.OpenXml.Wordprocessing;
using Pres = DocumentFormat.OpenXml.Presentation;

namespace Rag.Core.Corpus;

/// <summary>
/// Pulls plain text out of PDF, Word and PowerPoint files: the .NET stand-in for Docling.
///   PDF        -> PdfPig       (Apache 2.0)
///   DOCX, PPTX -> Open XML SDK (MIT)
/// The output is written as .txt into the corpus folder so the ingestion step can treat
/// every source the same way (the course never connected Docling to its ingestion script).
/// </summary>
public static class DocumentTextExtractor
{
    public static readonly string[] SupportedExtensions = [".pdf", ".docx", ".pptx", ".txt", ".md"];

    public static string ExtractText(string path) => Path.GetExtension(path).ToLowerInvariant()
switch
```

```
    {
        ".pdf" => FromPdf(path),
        ".docx" => FromDocx(path),
        ".pptx" => FromPptx(path),
        ".txt" or ".md" => File.ReadAllText(path),
        var ext => throw new NotSupportedException($"File type '{ext}' is not supported.")
    };

    /// <summary>Extracts one file (or every supported file in a folder) into <paramref
name="outputFolder"/>.</summary>
    public static int ExtractToFolder(string input, string outputFolder)
    {
        Directory.CreateDirectory(outputFolder);
        var files = Directory.Exists(input)
            ? Directory.EnumerateFiles(input).Where(f =>
SupportedExtensions.Contains(Path.GetExtension(f).ToLowerInvariant()))
            : [input];

        int count = 0;
        foreach (var file in files)
        {
            try
            {
                var text = ExtractText(file);
                var name =
WikipediaCorpusGenerator.SanitizeFileName(Path.GetFileNameWithoutExtension(file)) + ".txt";
                File.WriteAllText(Path.Combine(outputFolder, name), text);
                Console.WriteLine($"  {Path.GetFileName(file)} -> {name} ({text.Length:N0} chars)");
                count++;
            }
            catch (Exception ex)
            {
                Console.WriteLine($"  Error extracting {file}: {ex.Message}");
            }
        }
        return count;
    }

    /// <summary>PDF: read the words on each page; one paragraph per page.</summary>
    private static string FromPdf(string path)
    {
        var sb = new StringBuilder();
        using var pdf = PdfDocument.Open(path);
        foreach (var page in pdf.GetPages())
        {
            // page.Text often glues words together; joining the detected words is more reliable.
            sb.AppendLine(string.Join(" ", page.GetWords().Select(w => w.Text)));
            sb.AppendLine();
        }
        return sb.ToString();
    }

    /// <summary>DOCX: paragraphs and tables, in document order. Table rows become "cell | cell |
cell".</summary>
    private static string FromDocx(string path)
    {
        var sb = new StringBuilder();
        using var doc = WordprocessingDocument.Open(path, false);
        var body = doc.MainDocumentPart?.Document?.Body;
        if (body is null) return "";

        foreach (var element in body.ChildElements)
        {
            switch (element)
            {
                case Word.Paragraph p:
                    if (!string.IsNullOrWhiteSpace(p.InnerText))
```

```
sb.AppendLine(p.InnerText).AppendLine();
                    break;
                case Word.Table table:
                    foreach (var row in table.Elements<Word.TableRow>())
                    {
                        var cells = row.Elements<Word.TableCell>().Select(c => c.InnerText.Trim());
                        sb.AppendLine(string.Join(" | ", cells) + ".");
                    }
                    sb.AppendLine();
                    break;
            }
        }
        return sb.ToString();
    }

    /// <summary>PPTX: text of every slide, in slide order.</summary>
    private static string FromPptx(string path)
    {
        var sb = new StringBuilder();
        using var doc = PresentationDocument.Open(path, false);
        var presentationPart = doc.PresentationPart;
        var slideIds = presentationPart?.Presentation?.SlideIdList?.Elements<Pres.SlideId>();
        if (presentationPart is null || slideIds is null) return "";

        int number = 0;
        foreach (var slideId in slideIds)
        {
            number++;
            var relId = slideId.RelationshipId?.Value;
            if (relId is null) continue;
            var slidePart = (SlidePart)presentationPart.GetPartById(relId);

            var lines = slidePart.Slide?.Descendants<Drawing.Paragraph>()
                .Select(p => p.InnerText.Trim())
                .Where(t => t.Length > 0) ?? [];

            sb.AppendLine($"Slide {number}.").AppendLine();
            // A blank line after each text line keeps bullet points as separate "sentences".
            foreach (var line in lines) sb.AppendLine(line).AppendLine();
        }
        return sb.ToString();
    }
}
```

src/Rag.Cli/Program.cs: Extract()

```
void Extract()
{
    var input = args.Length > 1 ? args[1] : Path.Combine("data", "my_docs");
    Console.WriteLine($"Extracting '{input}' -> '{settings.CorpusFolder}'");
    var n = DocumentTextExtractor.ExtractToFolder(input, settings.CorpusFolder);
    Console.WriteLine($"{n} file(s) extracted.");
}
```

Put a few of your own files in `data/my_docs` and run:

```
dotnet run --project src/Rag.Cli -- extract data/my_docs
```

WATCH OUT

Scanned PDFs are just pictures of text, so PdfPig finds no words in them. Those need OCR (optical character recognition) first, for example with the open-source Tesseract. Check the generated `.txt` file before ingesting.

CHECKPOINT

- `ollama list` shows `granite4.1:3b`, `qwen3-embedding:0.6b` and `rag-granite`.
- `docker ps` shows `rag-postgres` running, and `\dx` lists `vector`.
- `rag selftest` passes, and `rag check` reports both models and PostgreSQL.
- `rag similarity` ranks the near-synonyms highest.
- `data/all_articles` contains roughly 100 clean `.txt` files.

PHASE 3

The Vector Database

IN THIS PHASE

Course chapter 4 (“Setting up a database and retrieving vectors and files”). You will define the database table with EF Core and pgvector, split documents into sentences, embed and store every sentence (the offline half of RAG), and run your first similarity searches.

3.1 Embeddings in practice

Phase 1 explained what embeddings are. Three practical facts matter now:

1. **The size is fixed per model.** `qwen3-embedding:0.6b` always returns 1,024 numbers, `nomic-embed-text` returns 768, and the course’s SFR-Embedding-Mistral returned 4,096. The database column must be declared with the right size, and you can never mix vectors from different models in one table.
2. **Use the same model for documents and questions.** A question embedded with model A can’t be compared with documents embedded with model B. Their coordinate systems are unrelated.
3. **Use a dedicated embedding model.** The course first embedded with its chat model, then switched to a dedicated embedding model because search quality was much better. We use a dedicated model from the start.

COMPARED WITH THE PYTHON COURSE

The course’s switch to SFR-Embedding-Mistral (a 7-billion-parameter, roughly 14 GB model, licensed for non-commercial use only) would be very slow on a laptop CPU. The course later recommended a small Qwen embedding model instead, and that’s what we use: Qwen3-Embedding 0.6B, Apache 2.0, 1,024 dimensions. It also stays under pgvector’s 2,000-dimension limit for indexes, which matters in Phase 5.

3.2 The table

Each row holds one sentence: which file it came from, its position in that file, its text and its vector.

table text_embeddings

id bigint	file_name text	sentence_number int	content text	embedding vector(1024)
1	Human_rights_in_Germany.txt	0	Human rights in Germany are...	[0.03,-0.41,0.77,...]
2	Human_rights_in_Germany.txt	1	The Federal Constitutional...	[0.11,-0.08,0.52,...]
3	Human_rights_in_Germany.txt	2	Germany ratified the...	[-0.27,0.33,0.19,...]
...	...	...	...	...
812	Amnesty_International.txt	0	Amnesty International is...	[0.45,0.02,-0.36,...]

index on (file_name, sentence_number) → fast "give me sentences 10-16 of file X"

The text_embeddings table. The index on (file_name, sentence_number) makes fetching neighbouring sentences fast.

The entity class is a plain C# class. `Vector` comes from the Pgvector package:

src/Rag.Core/Data/TextEmbedding.cs

```
using Pgvector;

namespace Rag.Core.Data;

/// <summary>
/// One row in the vector database = one sentence from one file.
/// (This is the C# version of the course's SQLAlchemy TextEmbedding class.)
/// </summary>
public class TextEmbedding
{
    /// <summary>Primary key.</summary>
    public long Id { get; set; }

    /// <summary>Which .txt file the sentence came from, e.g. "Human_rights_in_Germany.txt".
</summary>
    public string FileName { get; set; } = "";

    /// <summary>Position of the sentence inside its file: 0, 1, 2, ...  Used to fetch neighbours.
</summary>
    public int SentenceNumber { get; set; }

    /// <summary>The sentence text itself.</summary>
    public string Content { get; set; } = "";

    /// <summary>The embedding: a list of numbers (1024 of them for qwen3-embedding:0.6b).</summary>
    public Vector Embedding { get; set; } = null!;
}
```

The `DbContext` maps it to PostgreSQL. Three details to notice:

- `UseVector()` teaches Npgsql how to send and receive the `vector` type.
- `HasColumnType("vector(1024)")` takes the size from `RagSettings`, so it always matches the embedding model.
- Explicit snake_case column names (`file_name`, `sentence_number`) keep the raw SQL we write in Phase 5 readable.

src/Rag.Core/Data/RagDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using Pgvector.EntityFrameworkCore;
```

```
namespace Rag.Core.Data;

/// <summary>
/// EF Core gateway to PostgreSQL + pgvector.
/// </summary>
public class RagDbContext(RagSettings settings) : DbContext
{
    public DbSet<TextEmbedding> TextEmbeddings => Set<TextEmbedding>();

    protected override void OnConfiguring(DbContextOptionsBuilder options) =>
        // UseVector() teaches Npgsql how to send/receive the pgvector "vector" type.
        options.UseNpgsql(settings.ConnectionString, npgsql => npgsql.UseVector());

    protected override void OnModelCreating(ModelBuilder model)
    {
        // CREATE EXTENSION IF NOT EXISTS vector;
        model.HasPostgresExtension("vector");

        model.Entity<TextEmbedding>(e =>
        {
            // Explicit snake_case names keep the raw SQL in Phase 5 simple and readable.
            e.ToTable("text_embeddings");
            e.HasKey(x => x.Id);
            e.Property(x => x.Id).HasColumnName("id");
            e.Property(x => x.FileName).HasColumnName("file_name").IsRequired();
            e.Property(x => x.SentenceNumber).HasColumnName("sentence_number");
            e.Property(x => x.Content).HasColumnName("content").IsRequired();

            // vector(1024): the size MUST match the embedding model's output size.
            e.Property(x => x.Embedding)
             .HasColumnName("embedding")
             .HasColumnType($"vector({settings.EmbeddingDimensions})");

            // Fast lookup of "sentences 10..16 of file X" when we build context windows.
            e.HasIndex(x => new { x.FileName, x.SentenceNumber });
        });
    }
}
```

NOTE

Why `EnsureCreated()` and not migrations? EF Core migrations are the right tool for evolving production schemas. For a learning project with one table, `Database.EnsureCreatedAsync()` (called by the ingestor) creates the table on the first run with no extra tooling. If you change the schema, run `docker compose down -v` and `docker compose up -d` for a fresh database.

COMPARED WITH THE PYTHON COURSE

The course's SQLAlchemy model has the same five columns. In the video, though, the upload loop only zips embeddings with sentences and never fills `file_name` or `sentence_number`. Neighbour lookup in chapter 5 depends on those two columns, so here they are always stored.

3.3 Splitting text into sentences

The course uses NLTK's `sent_tokenize`. .NET has no direct equivalent, so we write a compact rule-based splitter. "Split at every full stop" is wrong surprisingly often, so the splitter handles the common traps:

Input	Naive split	Our splitter
`Dr. Smith moved to the U.S. in 1990.`	3 pieces	1 sentence (known abbreviations)
`J. R. R. Tolkien wrote books.`	4 pieces	1 sentence (single-letter initials)
`He earned $3.5 million!`	split at “3.”	1 sentence (a full stop must be followed by a space)
`== History ==`	a “sentence”	dropped (Wikipedia heading)
a line wrapped in the middle	2 fragments	re-joined (a line break only ends a sentence after `.`, `!`, `?` or `:`)

src/Rag.Core/Text/SentenceSplitter.cs

```
using System.Text;
using System.Text.RegularExpressions;

namespace Rag.Core.Text;

/// <summary>
/// Splits text into sentences: the C# stand-in for NLTK's sent_tokenize.
/// A rule-based splitter is not perfect, but it handles normal prose,
/// common abbreviations ("Dr.", "e.g.", "U.S."), initials ("J. Smith"),
/// decimal numbers ("3.5"), headings and hard-wrapped lines.
/// </summary>
public static partial class SentenceSplitter
{
    // Abbreviations that end with a dot but do NOT end a sentence.
    private static readonly HashSet<string> Abbreviations = new(StringComparer.OrdinalIgnoreCase)
    {
        "mr", "mrs", "ms", "dr", "prof", "sr", "jr", "st", "mt", "vs", "etc", "e.g", "i.e",
        "jan", "feb", "mar", "apr", "jun", "jul", "aug", "sep", "sept", "oct", "nov", "dec",
        "no", "vol", "fig", "approx", "inc", "ltd", "co", "corp", "u.s", "u.k", "u.n", "e.u",
    };

    /// <param name="minLength">Fragments shorter than this (stray headings, page numbers) are
dropped.</param>
    public static List<string> Split(string text, int minLength = 15)
    {
        var sentences = new List<string>();
        if (string.IsNullOrWhiteSpace(text)) return sentences;

        foreach (var paragraph in ToParagraphs(text))
        {
            // Inside a paragraph, look at every ". ", "! " or "? " followed by a likely sentence
start.
            int start = 0;
            foreach (Match m in CandidateBoundary().Matches(paragraph))
            {
                if (paragraph[m.Index] == '.' && IsAbbreviation(paragraph, m.Index)) continue;
                int boundary = m.Index + m.Length;          // includes punctuation, closing quote
and spaces
                Add(sentences, paragraph[start..boundary], minLength);
                start = boundary;
            }
            Add(sentences, paragraph[start..], minLength);
        }
        return sentences;
    }

    /// <summary>
```

```
    /// Groups lines into paragraphs. A line break ends a paragraph when the line ends a sentence,
    /// when a blank line follows, or when the next line is a heading or bullet. Otherwise the
    /// line was just wrapped and gets joined to the next one.
    /// </summary>
    private static IEnumerable<string> ToParagraphs(string text)
    {
        var current = new StringBuilder();
        foreach (var raw in text.Replace("\r\n", "\n").Split('\n'))
        {
            var line = Whitespace().Replace(raw, " ").Trim();

            if (line.Length == 0 || IsHeading(line) || IsBullet(line))
            {
                if (current.Length > 0) { yield return current.ToString().Trim(); current.Clear(); }
                if (IsBullet(line)) yield return line.TrimStart('-', '*', '•', ' ');
                continue;   // blank lines and headings ("== History ==") are dropped
            }

            current.Append(line).Append(' ');
            if (".!?:\"”')".Contains(line[^1]))
            {
                yield return current.ToString().Trim();
                current.Clear();
            }
        }
        if (current.Length > 0) yield return current.ToString().Trim();
    }

    private static bool IsHeading(string line) => line.StartsWith('=') || line.StartsWith('#');
    private static bool IsBullet(string line) => line.StartsWith("- ") || line.StartsWith("* ") ||
line.StartsWith('•');

    private static void Add(List<string> list, string candidate, int minLength)
    {
        var s = candidate.Trim();
        if (s.Length >= minLength) list.Add(s);
    }

    /// <summary>True when the word right before the dot is a known abbreviation or a single initial
("J. Smith").</summary>
    private static bool IsAbbreviation(string text, int dotIndex)
    {
        int i = dotIndex - 1;
        while (i >= 0 && (char.IsLetter(text[i]) || text[i] == '.')) i--;
        var word = text[(i + 1)..dotIndex];
        if (word.Length == 0) return false;
        if (word.Length == 1 && char.IsUpper(word[0])) return true;   // initials: "J. R. R.
Tolkien"
        return Abbreviations.Contains(word);
    }

    [GeneratedRegex(@"\s+")]
    private static partial Regex Whitespace();

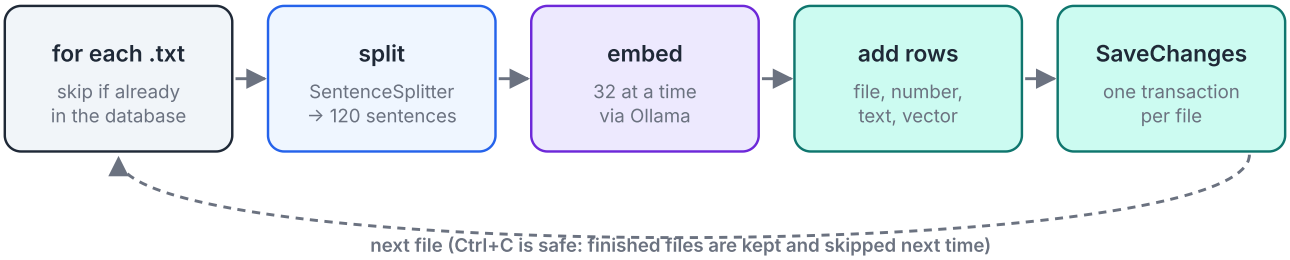
    // ". " / "! " / "? " (optionally followed by a closing quote/bracket) then an upper-case
letter, digit or opening quote.
    [GeneratedRegex(@"[.!?][""'”’)\]]?\s+(?=[A-Z0-9""“‘(\[])")]
    private static partial Regex CandidateBoundary();
}
```

`rag selftest` contains checks for exactly these cases. Run it after any change to the splitter.

> **TIP**
>
> **Regex source generators.** `[GeneratedRegex]` makes the compiler generate the regex matching code at build time. It is faster than `new Regex(...)` and catches pattern mistakes at compile time.

3.4 Ingestion: fill the database

Ingestion is the offline half of RAG. For each `.txt` file: split it into sentences, embed them, and store one row per sentence.



Embedding is >90% of the run time. ~100 articles ≈ 10k sentences ≈ 15-40 min on your laptop CPU

The ingestion loop. One transaction per file means an interrupted run loses at most the file it was working on.

src/Rag.Core/Ingestion/Ingestor.cs

```
using System.Diagnostics;
using Microsoft.EntityFrameworkCore;
using Pgvector;
using Rag.Core.Ai;
using Rag.Core.Data;
using Rag.Core.Text;

namespace Rag.Core.Ingestion;

/// <summary>
/// Fills the vector database: the C# version of the course's populate_vector_db.py.
/// For every .txt file: split into sentences -> embed each sentence -> store (file, position, text,
vector).
///
/// Improvements over the course version:
///   * Resumable: files already in the database are skipped, so a crash or Ctrl+C costs nothing.
///   * No duplicates: re-running does not insert the same sentences twice (use --reset to start
over).
///   * file_name and sentence_number are always stored (the course's video forgot them).
/// </summary>
public sealed class Ingestor(RagDbContext db, EmbeddingService embeddings, RagSettings settings)
{
    public async Task IngestFolderAsync(string folder, bool reset = false, int? maxFiles = null,
CancellationToken ct = default)
    {
        await db.Database.EnsureCreatedAsync(ct);   // creates the extension + table on first run

        if (reset)
        {
            await db.Database.ExecuteSqlRawAsync("TRUNCATE TABLE text_embeddings RESTART IDENTITY;",
ct);
            Console.WriteLine("Table emptied (--reset).");
```

```
        }

        var alreadyDone = (await db.TextEmbeddings.Select(e =>
e.FileName).Distinct().ToListAsync(ct)).ToHashSet();

        var files = Directory.EnumerateFiles(folder, "*.txt").OrderBy(f => f).ToList();
        if (maxFiles is int max) files = files.Take(max).ToList();
        Console.WriteLine($"{files.Count} file(s) in '{folder}', {alreadyDone.Count} already
ingested.");

        var total = Stopwatch.StartNew();
        int fileNo = 0, sentenceCount = 0;

        foreach (var path in files)
        {
            fileNo++;
            var fileName = Path.GetFileName(path);
            if (alreadyDone.Contains(fileName)) continue;

            try
            {
                var timer = Stopwatch.StartNew();
                var sentences = SentenceSplitter.Split(await File.ReadAllTextAsync(path, ct));
                if (sentences.Count == 0) { Console.WriteLine($"[{fileNo}/{files.Count}] {fileName}:
no sentences, skipped"); continue; }

                // The slow part: one embedding per sentence (sent to Ollama in batches).
                var vectors = await embeddings.EmbedDocumentsAsync(sentences,
settings.EmbeddingBatchSize, ct);

                for (int i = 0; i < sentences.Count; i++)
                {
                    db.TextEmbeddings.Add(new TextEmbedding
                    {
                        FileName = fileName,
                        SentenceNumber = i,
                        Content = sentences[i],
                        Embedding = new Vector(vectors[i]),
                    });
                }

                await db.SaveChangesAsync(ct);   // one transaction per file
                db.ChangeTracker.Clear();         // keep memory flat across hundreds of files

                sentenceCount += sentences.Count;
                Console.WriteLine($"[{fileNo}/{files.Count}] {fileName}: {sentences.Count} sentences
in {timer.Elapsed.TotalSeconds:F1}s");
            }
            catch (Exception ex) when (ex is not OperationCanceledException)
            {
                db.ChangeTracker.Clear();
                Console.WriteLine($"[{fileNo}/{files.Count}] Error processing {fileName}:
{ex.Message}");
            }
        }

        Console.WriteLine($"Finished: {sentenceCount:N0} new sentences in
{total.Elapsed:hh\\:mm\\:ss}.");
    }
}
```

What the code does, step by step:

1. `EnsureCreatedAsync` creates the table on the first run.

2. `--reset` empties the table (`TRUNCATE`) when you want to start over, for example after changing the embedding model.
3. It loads the names of files already in the table and **skips them**, so you can stop with Ctrl+C at any time and simply run it again.
4. For each new file: split, embed in batches of 32, add the rows, then `SaveChangesAsync`, which inserts the whole file in one transaction.
5. `ChangeTracker.Clear()` stops EF Core from keeping thousands of finished entities in memory.

src/Rag.Cli/Program.cs: IngestAsync()

```
async Task IngestAsync()
{
    await using var db = new RagDbContext(settings);
    var embeddings = new EmbeddingService(AiClients.CreateEmbeddingGenerator(settings));
    var maxFiles = Option("--max-files") is { } m ? int.Parse(m) : (int?)null;
    await new Ingestor(db, embeddings, settings).IngestFolderAsync(settings.CorpusFolder, Flag("--
reset"), maxFiles, ct);
}
```

Run it

Test with a handful of files first, then run the whole corpus:

```
dotnet run --project src/Rag.Cli -- ingest --max-files 3
dotnet run --project src/Rag.Cli -- ingest
```

```
98 file(s) in 'data\all_articles', 3 already ingested.
[4/98] Amnesty_International.txt: 211 sentences in 18.4s
[5/98] Convention_on_the_Rights_of_the_Child.txt: 143 sentences in 12.9s
...
Finished: 10,4xx new sentences in 00:2x:xx.
```

ON YOUR LAPTOP

Embedding takes more than 90% of the time. On your i7 CPU, expect very roughly 5–15 sentences per second. That's about 15–40 minutes for 100 articles, and less if the GTX 960M is in use (`ollama ps` while it runs). Leave it running in its own terminal. Because the ingestor is resumable, the course's `screen` trick isn't needed: if anything stops it, run the same command again.

Check the result in the database:

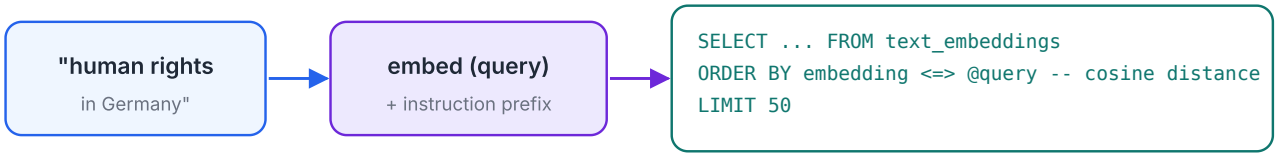
```
docker exec -it rag-postgres psql -U postgres -d text_embeddings `
  -c "SELECT file_name, count(*) FROM text_embeddings GROUP BY file_name ORDER BY 2 DESC LIMIT 5;"
```

COMPARED WITH THE PYTHON COURSE

The course's script, when run twice, inserts every sentence twice, and a crash halfway through means starting again. It also embeds with the chat model first and is then interrupted to switch models. Here the embedding model is right from the start, files are skipped once stored, and each file is saved in one transaction.

3.5 Vector search

Now the online half begins. To search, embed the question (with the instruction prefix from Phase 2) and ask PostgreSQL for the closest sentences:



Result: a ranked list of sentences (smaller distance = closer meaning)

```
0.21    Human_rights_in_Germany.txt #0
0.27    Human_rights_in_Germany.txt #14
0.31    Basic_Law_for_the_Federal_Republic... #3
```

From question to ranked sentences. EF Core translates CosineDistance() into pgvector's ⇔ operator.

src/Rag.Core/Retrieval/Retriever.cs: VectorSearchAsync

```
/// <summary>
/// Exact top-k vector search. EF Core translates CosineDistance() into pgvector's
/// "&lt;=&gt;" operator:  SELECT ... ORDER BY embedding &lt;=&gt; @query LIMIT @k
/// </summary>
public async Task<List<SearchHit>> VectorSearchAsync(float[] queryVector, int k, CancellationToken
ct = default)
{
    await EnsureSessionSettingsAsync(ct);
    var q = new Vector(queryVector);
    return await db.TextEmbeddings
        .AsNoTracking()
        .OrderBy(e => e.Embedding.CosineDistance(q))
        .Take(k)
        .Select(e => new SearchHit(e.Id, e.FileName, e.SentenceNumber, e.Content,
e.Embedding.CosineDistance(q)))
        .ToListAsync(ct);
}
```

`e.Embedding.CosineDistance(q)` looks like C#, but EF Core translates the whole query into SQL, so the database does the work:

```
SELECT t.id, t.file_name, t.sentence_number, t.content, t.embedding <=> @q
FROM text_embeddings AS t
ORDER BY t.embedding <=> @q
LIMIT @k
```

(`VectorSearchAsync` also calls a small `EnsureSessionSettingsAsync` helper. It only matters once you add an index in Phase 5, and is explained there.)

src/Rag.Cli/Program.cs: SearchAsync()

```
async Task SearchAsync()
{
    var query = RequireText("search");
    var k = int.Parse(Option("--k") ?? "5");
    await using var db = new RagDbContext(settings);
    var embeddings = new EmbeddingService(AiClients.CreateEmbeddingGenerator(settings));
```

```
    var retriever = new Retriever(db, embeddings, settings);

    var hits = await retriever.VectorSearchAsync(await embeddings.EmbedQueryAsync(query, ct), k,
ct);
    Console.WriteLine($"Top {hits.Count} sentences for: {query}\n");
    foreach (var h in hits)
        Console.WriteLine($"  {h.Score:F3}  {h.FileName} #{h.SentenceNumber}\n
{h.Content}\n");
}
```

```
dotnet run --project src/Rag.Cli -- search "human rights in Germany"
```

```
Top 5 sentences for: human rights in Germany

  0.2xx  Human_rights_in_Germany.txt #0
         Human rights in Germany are protected by the Basic Law ...
  0.2xx  ...
```

Smaller distances mean closer meanings. Try a few searches that use different words from the documents ("freedom of speech" versus "censorship", for example). Vector search finds matching *meaning*, not just matching words, and that is its big advantage over classic keyword search.

3.6 From sentences to context

A single sentence is often too little for the LLM to work with. "It was ratified in 1952." is useless without knowing what *it* is. So, as the course explains, each match is expanded into a **window**: the matching sentence plus `WindowSize` (3) sentences before and after it, taken from the same file. That's why we stored `sentence_number`:

```
db.TextEmbeddings
  .Where(e => e.FileName == file && e.SentenceNumber >= n - 3 && e.SentenceNumber <= n + 3)
  .OrderBy(e => e.SentenceNumber)
```

Windows raise one new problem. When two matches sit close together, their windows **overlap**, and the model would read the same sentences twice. Solving that cleanly is the first job of Phase 4.

COMPARED WITH THE PYTHON COURSE

The course's chapter 4 version (`get_surrounding_sentences`) looks neighbours up by the row `id` instead of `sentence_number` . That only works because each file's rows happen to be inserted in one go. Using the file name plus the sentence number, backed by an index, is correct no matter how rows were inserted.

CHECKPOINT

- `rag ingest` finished, and running it again reports every file as already ingested.
- The `GROUP BY` query shows rows for your files.
- `rag search` returns sentences that clearly relate to your query, with distances well below 1.
- You can explain why the question and the documents must use the same embedding model.

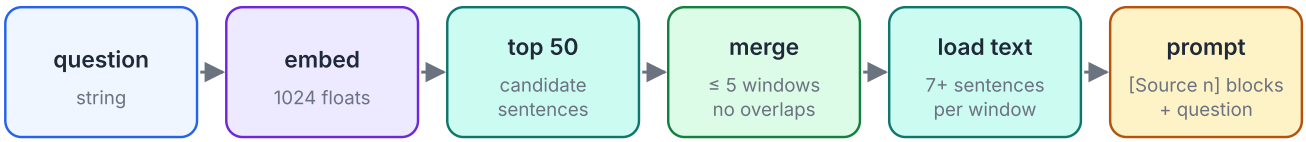
PHASE 4

The Full Pipeline and Prompt Engineering

IN THIS PHASE

Course chapter 5 (“Packaging parts, pipeline engineering, and prompt engineering”). You will turn search hits into clean, non-overlapping context windows, design the prompt (static rules in the Modelfile, dynamic context from C#), and stream grounded answers with citations from the `rag ask` command.

4.1 The retrieval pipeline



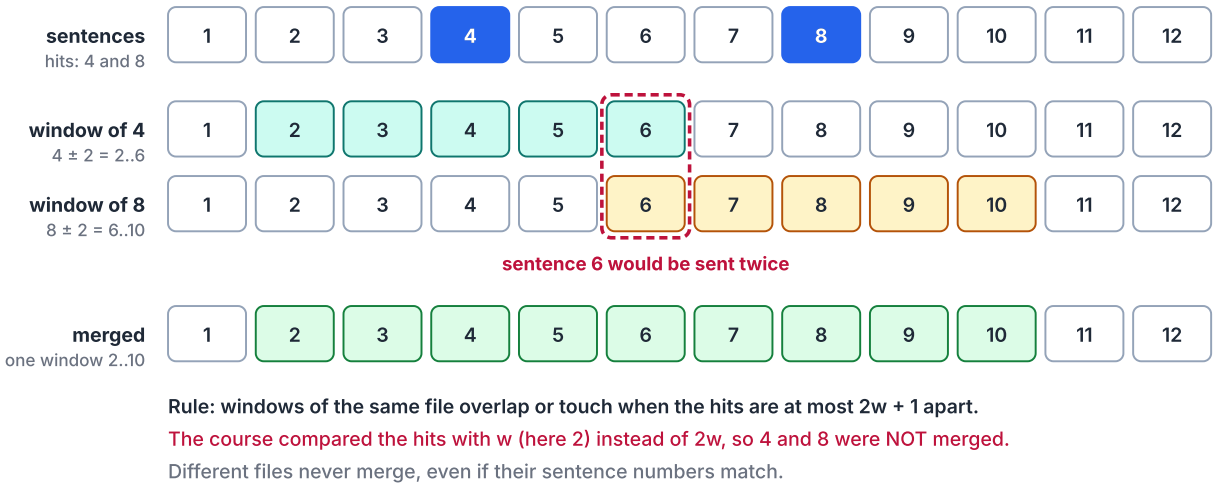
Why 50 candidates for 5 windows? Many top hits sit next to each other and merge into one window.

From a question to a finished prompt. Each box is a few lines of code in Retriever and PromptBuilder.

The plan, as in the course: fetch **many more** candidate sentences than we need (50), merge the ones that overlap, keep the best **5 distinct windows**, load their text, and format the prompt.

4.2 Merging overlapping windows

Picture a file with 12 sentences, where the search matches sentences **4** and **8**, and the window size is ±2:



Rule: windows of the same file overlap or touch when the hits are at most 2w + 1 apart.

The course compared the hits with w (here 2) instead of 2w, so 4 and 8 were NOT merged.

Different files never merge, even if their sentence numbers match.

Two windows that share sentence 6 are merged into one continuous window. This is the course’s own example, with its off-by-one corrected.

The windows 2–6 and 6–10 share sentence 6. Sending both would waste context space and show the model a duplicate. Instead we merge them into a single window, 2–10, with one citation.

When do two windows overlap? A hit at `n` covers `[n − w, n + w]`. Two such ranges in the same file meet when the hits are at most `2w` apart, and they touch (with no gap between them) at `2w + 1`. We merge in both cases, so the model gets one continuous passage.

COMPARED WITH THE PYTHON COURSE

A bug in the course. The course's `is_unique_to_window` and `group_entries` treat two hits as overlapping only when they are within `w` of each other. In its own example (hits 4 and 8, w = 2) the gap is 4, more than 2, so they are *not* merged and sentence 6 is sent twice, which is exactly what the lesson set out to prevent. The course's version also needs three helper functions (`group_entries`, `consolidate_groupings`, `get_min_max_ids`). Treating each window as a range of sentence numbers reduces it all to one short loop.

The algorithm walks through the hits in rank order (best first):

1. Compute the hit's range `[n − w, n + w]`, starting at 0 at the beginning of a file.
2. Find existing windows **of the same file** that overlap or touch it.
3. None found: it's a new region. Add a window, unless we already have 5. In that case, stop.
4. Some found: extend the best-ranked window to cover the new range. If the new range bridges two windows, fold them all into one.

src/Rag.Core/Retrieval/WindowMerger.cs

```
namespace Rag.Core.Retrieval;

/// <summary>
/// A block of consecutive sentences from ONE file: sentences Start..End (inclusive).
/// </summary>
public sealed class ContextWindow
{
    public required string FileName { get; init; }
    public int Start { get; set; }
    public int End { get; set; }

    /// <summary>The search hits that produced this window, in rank order.</summary>
    public List<SearchHit> Hits { get; } = [];

    /// <summary>Filled in later with the actual sentence text.</summary>
    public List<string> Sentences { get; } = [];

    public override string ToString() => $"{FileName} [{Start}..{End}] ({Hits.Count} hit(s))";
}

/// <summary>
/// Turns a ranked list of matching sentences into a few non-overlapping context windows.
///
/// Each hit at sentence n wants the window [n - w, n + w].
/// Two windows in the same file overlap (or touch) when their ranges meet, i.e. when the hits
/// are at most 2w + 1 sentences apart. Overlapping windows are merged into one, so the LLM
/// never sees the same sentence twice. This fixes the course's off-by-one (it compared
/// the hits with w instead of 2w).
/// </summary>
public static class WindowMerger
{
    public static List<ContextWindow> SelectWindows(IEnumerable<SearchHit> rankedHits, int
```

```
windowSize, int maxWindows)
    {
        var windows = new List<ContextWindow>();

        foreach (var hit in rankedHits)
        {
            int start = Math.Max(0, hit.SentenceNumber - windowSize);
            int end = hit.SentenceNumber + windowSize;

            // Every existing window of the same file that overlaps or touches [start, end].
            var overlapping = windows
                .Where(w => w.FileName == hit.FileName && start <= w.End + 1 && end >= w.Start - 1)
                .ToList();

            if (overlapping.Count == 0)
            {
                // A genuinely new region of text.
                if (windows.Count == maxWindows) break;      // we already have enough distinct
windows
                var window = new ContextWindow { FileName = hit.FileName, Start = start, End = end
};
                window.Hits.Add(hit);
                windows.Add(window);
                continue;
            }

            // Merge the new range (and any windows it bridges) into the first, best-ranked window.
            var keep = overlapping[0];
            keep.Start = Math.Min(keep.Start, start);
            keep.End = Math.Max(keep.End, end);
            keep.Hits.Add(hit);
            foreach (var other in overlapping.Skip(1))
            {
                keep.Start = Math.Min(keep.Start, other.Start);
                keep.End = Math.Max(keep.End, other.End);
                keep.Hits.AddRange(other.Hits);
                windows.Remove(other);
            }
        }

        return windows;
    }
}
```

`rag selftest` checks every rule above: the 4-and-8 example, touching windows, distant hits, different files, a hit bridging two windows, the 5-window limit, and the start never going below sentence 0.

4.3 The Retriever, complete

`RetrieveWindowsAsync` ties it together: embed the question, fetch 50 candidates, merge them into windows, and load each window's sentences with one small indexed query. Here is the whole class. The keyword and hybrid search methods belong to Phase 5. You can skip them for now.

src/Rag.Core/Retrieval/Retriever.cs

```
using Microsoft.EntityFrameworkCore;
using Pgvector;
using Pgvector.EntityFrameworkCore;
using Rag.Core.Ai;
using Rag.Core.Data;

namespace Rag.Core.Retrieval;
```

```
/// <summary>
/// Finds the text that is relevant to a question.
/// Phase 3: plain vector search.  Phase 4: windows + context.  Phase 5: hybrid search.
/// </summary>
public sealed class Retriever(RagDbContext db, EmbeddingService embeddings, RagSettings settings)
{
    // ------------------------------------------------------------- Phase 3

    /// <summary>
    /// Exact top-k vector search. EF Core translates CosineDistance() into pgvector's
    /// "&lt;=&gt;" operator:  SELECT ... ORDER BY embedding &lt;=&gt; @query LIMIT @k
    /// </summary>
    public async Task<List<SearchHit>> VectorSearchAsync(float[] queryVector, int k,
CancellationToken ct = default)
    {
        await EnsureSessionSettingsAsync(ct);
        var q = new Vector(queryVector);
        return await db.TextEmbeddings
            .AsNoTracking()
            .OrderBy(e => e.Embedding.CosineDistance(q))
            .Take(k)
            .Select(e => new SearchHit(e.Id, e.FileName, e.SentenceNumber, e.Content,
e.Embedding.CosineDistance(q)))
            .ToListAsync(ct);
    }

    private bool _sessionReady;

    /// <summary>
    /// Once you add an HNSW index (Phase 5), pgvector returns at most hnsw.ef_search rows
    /// (default 40) per query, fewer than our 50 candidates. Raising it per session avoids that.
    /// We open the connection ourselves so EF Core keeps using the same one (and the setting
sticks).
    /// </summary>
    private async Task EnsureSessionSettingsAsync(CancellationToken ct)
    {
        if (_sessionReady) return;
        await db.Database.OpenConnectionAsync(ct);
        int efSearch = Math.Max(100, settings.CandidatePool * 2);   // a plain number, so no SQL-
injection risk
        await db.Database.ExecuteSqlRawAsync("SET hnsw.ef_search = " + efSearch, ct);
        _sessionReady = true;
    }

    // ------------------------------------------------------------- Phase 5

    /// <summary>
    /// Keyword (full-text) search with PostgreSQL's built-in text search.
    /// Great at exact names and rare terms, which embeddings sometimes miss.
    /// </summary>
    public async Task<List<SearchHit>> KeywordSearchAsync(string question, int k, CancellationToken
ct = default)
    {
        var rows = await db.Database.SqlQuery<KeywordRow>($"""
            SELECT id AS "Id", file_name AS "FileName", sentence_number AS "SentenceNumber", content
AS "Content",
                   ts_rank(to_tsvector('english', content), websearch_to_tsquery('english',
{question})) AS "Rank"
            FROM text_embeddings
            WHERE to_tsvector('english', content) @@ websearch_to_tsquery('english', {question})
            ORDER BY "Rank" DESC
            LIMIT {k}
            """).ToListAsync(ct);

        return rows.Select(r => new SearchHit(r.Id, r.FileName, r.SentenceNumber, r.Content,
r.Rank)).ToList();
```

```
    }

    /// <summary>Hybrid search: run vector + keyword search and fuse the two rankings with RRF.
</summary>
    public async Task<List<SearchHit>> HybridSearchAsync(string question, float[] queryVector, int
k, CancellationToken ct = default)
    {
        var vector = await VectorSearchAsync(queryVector, k, ct);
        var keyword = await KeywordSearchAsync(question, k, ct);
        return RankFusion.Fuse([vector, keyword]).Take(k).ToList();
    }

    // ------------------------------------------------------------------ Phase 4

    /// <summary>
    /// The full retrieval pipeline:
    /// question -> embedding -> many candidate hits -> merged, non-overlapping windows -> sentence
text.
    /// </summary>
    public async Task<List<ContextWindow>> RetrieveWindowsAsync(
        string question, bool hybrid = false, int? maxWindows = null, CancellationToken ct =
default)
    {
        var queryVector = await embeddings.EmbedQueryAsync(question, ct);

        var candidates = hybrid
            ? await HybridSearchAsync(question, queryVector, settings.CandidatePool, ct)
            : await VectorSearchAsync(queryVector, settings.CandidatePool, ct);

        var windows = WindowMerger.SelectWindows(candidates, settings.WindowSize, maxWindows ??
settings.MaxWindows);
        await LoadSentencesAsync(windows, ct);
        return windows;
    }

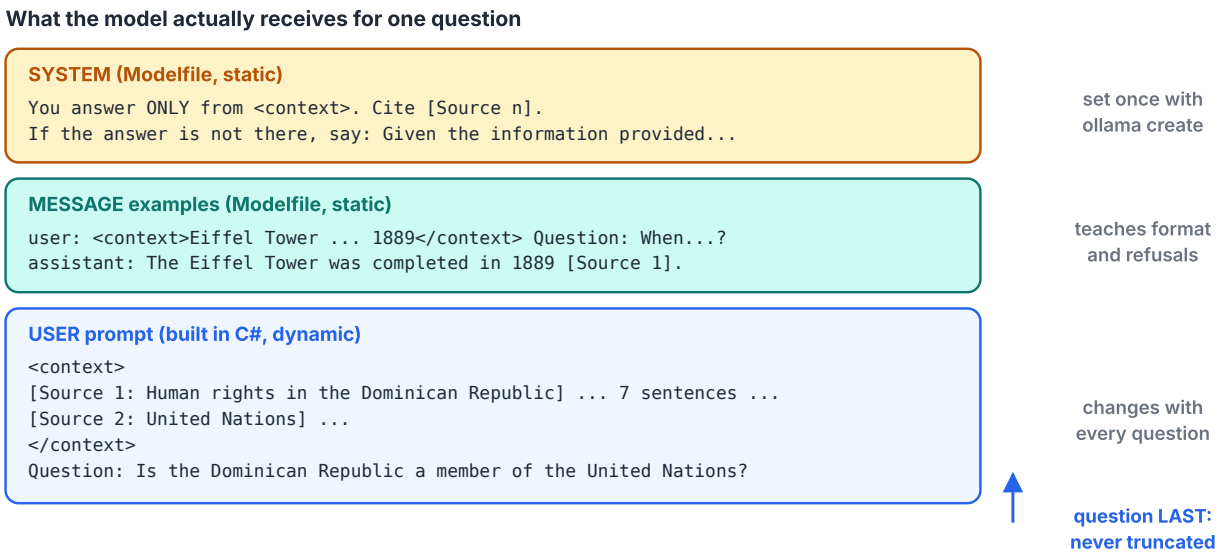
    /// <summary>Fetches the actual sentences for each window (one small indexed query per window).
</summary>
    public async Task LoadSentencesAsync(IEnumerable<ContextWindow> windows, CancellationToken ct =
default)
    {
        foreach (var w in windows)
        {
            var sentences = await db.TextEmbeddings
                .AsNoTracking()
                .Where(e => e.FileName == w.FileName && e.SentenceNumber >= w.Start &&
e.SentenceNumber <= w.End)
                .OrderBy(e => e.SentenceNumber)
                .Select(e => e.Content)
                .ToListAsync(ct);

            w.Sentences.Clear();
            w.Sentences.AddRange(sentences);
        }
    }
}

/// <summary>Shape of a keyword-search row (the SQL column aliases must match these property names).
</summary>
internal sealed class KeywordRow
{
    public long Id { get; set; }
    public string FileName { get; set; } = "";
    public int SentenceNumber { get; set; }
    public string Content { get; set; } = "";
    public float Rank { get; set; }
}
```

4.4 Prompt engineering

The course splits the prompt into two parts, and so do we:



The three parts of what the model receives. The Modelfile parts are fixed. The user prompt is built in C# for each question.

The static part: SYSTEM and MESSAGE in the Modelfile

Look again at the `Modelfile` from Phase 2. Its `SYSTEM` block gives the model four rules:

1. Use only facts stated in the context.
2. Cite the source label after each fact, for example `[Source 2]`.
3. If the context doesn't contain the answer, reply with a fixed sentence: *"Given the information provided, I'm unable to answer your question."*
4. Keep answers short.

The `MESSAGE` lines are **few-shot examples**: fake earlier turns of the conversation that *show* the model what we want. One example answers from the context with a citation. The other refuses because the context doesn't contain the answer. Examples like these often work better than instructions alone.

> **TIP**
>
> Rule 3 matters most. A fixed refusal sentence is easy to detect in code and in tests (Phase 5). It also stops the model from quietly falling back on its training knowledge when retrieval found nothing useful.

After editing the Modelfile, rebuild the model. It takes a second, because the weights are shared:

```
ollama create rag-granite -f Modelfile
```

Seeing what's under the hood

Ollama keeps each model's **chat template**, the code that inserts the special role tokens from Phase 1 around your messages:

```
ollama show --modelfile granite4.1:3b     # base model: TEMPLATE, stop tokens, licence
ollama show --modelfile rag-granite       # your model: base settings + your SYSTEM/MESSAGE lines
```

Your custom model inherits the template and stop tokens from the base model. You only add your own lines on top.

The dynamic part: context and question

The course's final prompt sent the context as raw database tuples, `(812, 3, 'The Dominican Republic...', 'Human_rights_in_the_Dominican_Republic.txt')`. The course itself points out that the IDs, numbers and punctuation are noise, and leaves the clean-up as an exercise. Here is that exercise, done:

src/Rag.Core/Generation/PromptBuilder.cs

```
using System.Text;
using Rag.Core.Retrieval;

namespace Rag.Core.Generation;

/// <summary>
/// Builds the DYNAMIC part of the prompt: retrieved context + the user's question.
/// The STATIC part (role, rules, examples) lives in the Modelfile's SYSTEM and MESSAGE lines.
///
/// This fixes the course's "raw tuples" problem: the LLM sees clean sentences grouped by
/// source with one numbered citation per block, never IDs, commas and parentheses.
/// </summary>
public static class PromptBuilder
{
    /// <summary>"Human_rights_in_Germany.txt" -> "Human rights in Germany".</summary>
    public static string SourceTitle(string fileName) =>
        Path.GetFileNameWithoutExtension(fileName).Replace('_', ' ').Trim();

    public static string FormatContext(IReadOnlyList<ContextWindow> windows)
    {
        var sb = new StringBuilder();
        for (int i = 0; i < windows.Count; i++)
        {
            sb.AppendLine($"[Source {i + 1}: {SourceTitle(windows[i].FileName)}]");
            sb.AppendLine(string.Join(" ", windows[i].Sentences));
            sb.AppendLine();
        }
        return sb.ToString().TrimEnd();
    }

    /// <summary>
    /// The question goes LAST: if a prompt is ever too long, Ollama cuts from the START,
    /// so the question must never be the part that gets cut.
    /// </summary>
    public static string BuildPrompt(string question, IReadOnlyList<ContextWindow> windows) =>
        $"""
        <context>
        {FormatContext(windows)}
        </context>

        Question: {question}
        """;
}
```

The prompt for one question looks like this:

```
<context>
[Source 1: Human rights in the Dominican Republic]
The Dominican Republic is a member of the United Nations ... (7+ sentences)

[Source 2: United Nations]
...
</context>

Question: Is the Dominican Republic a member of the United Nations?
```

Three deliberate choices:

- **Clean sentences grouped by source**, with one readable label per block, instead of raw rows.
- **Numbered labels** (`[Source 1]`) that match the source list printed for the user, so every citation can be checked.
- **The question last.** If a prompt is ever longer than `num_ctx`, Ollama cuts tokens from the start, so the question is never the part that's lost.

4.5 Generating the answer

`RagAnswerer` is the C# version of the course's `run_rag.py` : retrieve, augment, generate. Generation **streams**: `GetStreamingResponseAsync` returns tokens as the model produces them, so the user sees the answer appear immediately instead of waiting for all of it. That makes a big difference to how the app feels on a CPU.

src/Rag.Core/Generation/RagAnswerer.cs

```
using System.Runtime.CompilerServices;
using Microsoft.Extensions.AI;
using Rag.Core.Retrieval;

namespace Rag.Core.Generation;

/// <summary>Everything retrieval produced for one question.</summary>
public sealed record RagContext(string Question, List<ContextWindow> Windows, string Prompt)
{
    /// <summary>Numbered source list, matching the [Source n] labels in the prompt.</summary>
    public IEnumerable<string> Sources =>
        Windows.Select((w, i) => $"[Source {i + 1}] {PromptBuilder.SourceTitle(w.FileName)}
(sentences {w.Start}-{w.End})");
}

public enum RetrievalMode { Vector, Hybrid, HybridRerank }

/// <summary>
/// The complete RAG pipeline: the C# version of the course's run_rag.py.
///   1. retrieve: question -> context windows
///   2. augment:  windows + question -> prompt
///   3. generate: prompt -> streamed answer from the LLM
/// </summary>
public sealed class RagAnswerer(Retriever retriever, IChatClient chat, RagSettings settings,
LlmReranker? reranker = null)
{
    public async Task<RagContext> RetrieveAsync(string question, RetrievalMode mode =
RetrievalMode.Vector, CancellationToken ct = default)
    {
        bool rerank = mode == RetrievalMode.HybridRerank && reranker is not null;

        // When re-ranking, fetch twice as many windows, let the LLM sort them, keep the best ones.
```

```
        var windows = await retriever.RetrieveWindowsAsync(
            question,
            hybrid: mode != RetrievalMode.Vector,
            maxWindows: rerank ? settings.MaxWindows * 2 : settings.MaxWindows,
            ct: ct);

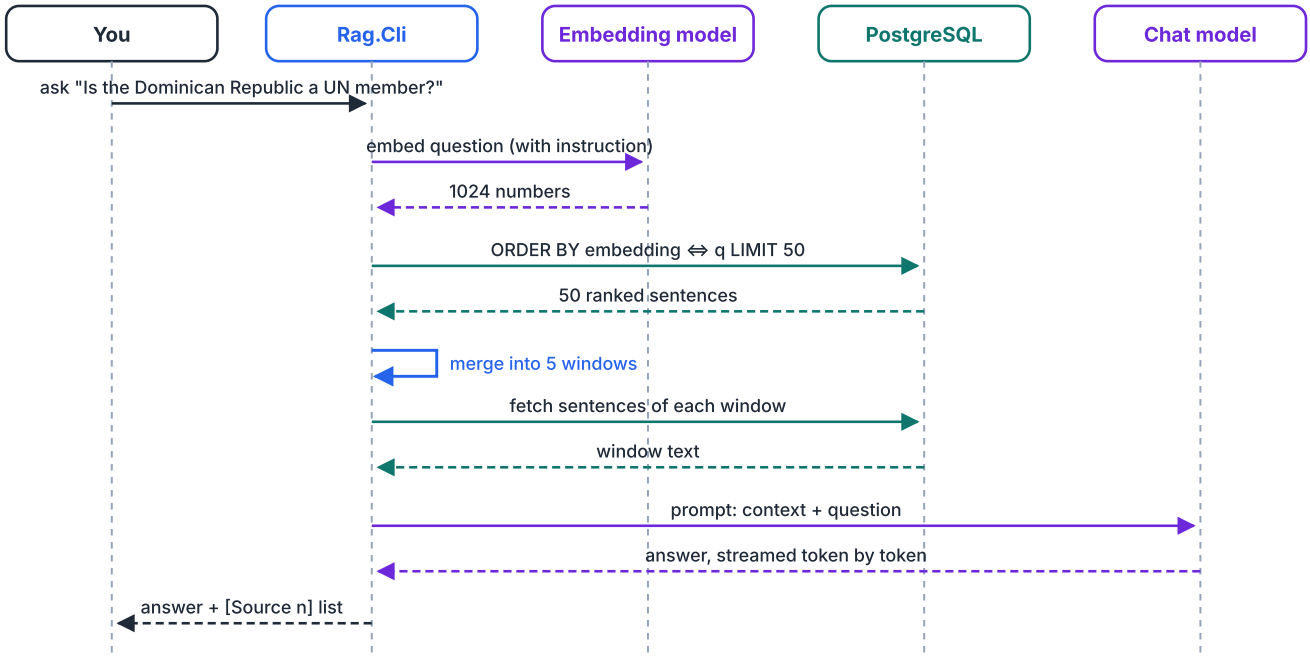
        if (rerank)
            windows = (await reranker!.RerankAsync(question, windows,
ct)).Take(settings.MaxWindows).ToList();

        return new RagContext(question, windows, PromptBuilder.BuildPrompt(question, windows));
    }

    /// <summary>Streams the answer token by token, so the user sees it appear immediately.
</summary>
    public async IAsyncEnumerable<string> AnswerAsync(RagContext context, [EnumeratorCancellation]
CancellationToken ct = default)
    {
        // Only the user message is sent: the system prompt + examples come from the Modelfile.
        await foreach (var update in chat.GetStreamingResponseAsync(context.Prompt,
cancellationToken: ct))
        {
            if (!string.IsNullOrEmpty(update.Text)) yield return update.Text;
        }
    }
}
```

NOTE

We send only a *user* message. The system prompt and examples come from the Modelfile. If you send a system message from code, it replaces the Modelfile's `SYSTEM` block.



The complete flow of one rag ask, from question to cited answer.

src/Rag.Cli/Program.cs: AskAsync()

```
async Task AskAsync()
{
```

```
    var question = RequireText("ask");
    await using var db = new RagDbContext(settings);
    var rag = CreateAnswerer(db);

    var sw = Stopwatch.StartNew();
    var context = await rag.RetrieveAsync(question, ParseMode(), ct);
    Console.WriteLine($"Retrieved {context.Windows.Count} context window(s) in
{sw.Elapsed.TotalSeconds:F1}s:");
    foreach (var s in context.Sources) Console.WriteLine("  " + s);

    if (Flag("--show-prompt"))
        Console.WriteLine($"\n----- PROMPT SENT TO THE LLM -----\n{context.Prompt}\n----------------
------------------");

    Console.WriteLine("\nAnswer:");
    sw.Restart();
    await foreach (var token in rag.AnswerAsync(context, ct)) Console.Write(token);
    Console.WriteLine($"\n\n({sw.Elapsed.TotalSeconds:F1}s)");
}
```

src/Rag.Cli/Program.cs: CreateAnswerer()

```
RagAnswerer CreateAnswerer(RagDbContext db)
{
    var embeddings = new EmbeddingService(AiClients.CreateEmbeddingGenerator(settings));
    var retriever = new Retriever(db, embeddings, settings);
    var reranker = new LlmReranker(AiClients.CreateChatClient(settings, settings.BaseChatModel));
    return new RagAnswerer(retriever, AiClients.CreateChatClient(settings), settings, reranker);
}
```

4.6 Run it

```
dotnet run --project src/Rag.Cli -- ask "Is the Dominican Republic a member of the United Nations?"
--show-prompt
```

```
Retrieved 5 context window(s) in 0.7s:
  [Source 1] Human rights in the Dominican Republic (sentences 0-9)
  [Source 2] ...

----- PROMPT SENT TO THE LLM -----
<context>
...
</context>

Question: Is the Dominican Republic a member of the United Nations?
---------------------------------

Answer:
Yes, the Dominican Republic is a member of the United Nations [Source 1].

(9.3s)
```

Now try questions the corpus *can't* answer, to test the refusal rule:

```
dotnet run --project src/Rag.Cli -- ask "Who won the 2018 football World Cup?"
# Given the information provided, I'm unable to answer your question.
```

ON YOUR LAPTOP

On your CPU, expect a few seconds before the first token, then a steady stream. Retrieval itself takes well under a second. Nearly all the time goes to generation. If a GPU is detected (`ollama ps`), generation will be noticeably faster.

4.7 When answers go wrong

The course warns that first results are often unimpressive and that prompt engineering is iterative. Always start by reading the prompt (`--show-prompt`): it tells you whether the problem is **retrieval** (wrong passages) or **generation** (right passages, bad answer).

Symptom	Likely cause	What to try
Answer is wrong and the right passage is missing from the prompt	Retrieval	Rephrase the question; check the article exists in `data/all_articles`; try `--mode hybrid` (Phase 5)
Right passage present, but the model refuses	Rule 3 too strict, or the fact is only implied	Loosen the wording ("if the context does not *reasonably* support an answer..."); add a few-shot example of an inferred answer
Answer uses facts not in the context	The model fell back on its own knowledge	Make rule 1 stronger; keep temperature 0; check the refusal example is in the Modelfile
No `[Source n]` citations	Small models forget format instructions	Add another few-shot example with citations; try `granite4.1:8b`
Very slow	Model on CPU, or a long prompt	Check `ollama ps`; lower `MaxWindows` or `WindowSize`; close other heavy apps
Answer cut off or context ignored	Prompt longer than `num_ctx`	Raise `num_ctx` in the Modelfile, or reduce windows

COMPARED WITH THE PYTHON COURSE

The course's final demo answered "Is the Dominican Republic a member of the United Nations?" with *"Yes, according to the text..."*, but most of its context was about an Inter-American Court ruling. With numbered sources and the citation rule, you can verify every claim against the source list instead of trusting the answer.

CHECKPOINT

- `rag ask` streams an answer with `[Source n]` citations that match the printed source list.
- Questions outside the corpus get the exact refusal sentence.
- With `--show-prompt` you can see clean, merged windows with no duplicated sentences.
- You can tell a retrieval problem from a generation problem by reading the prompt.

Congratulations: you have now rebuilt the entire course in .NET. Phase 5 goes beyond it.

PHASE 5

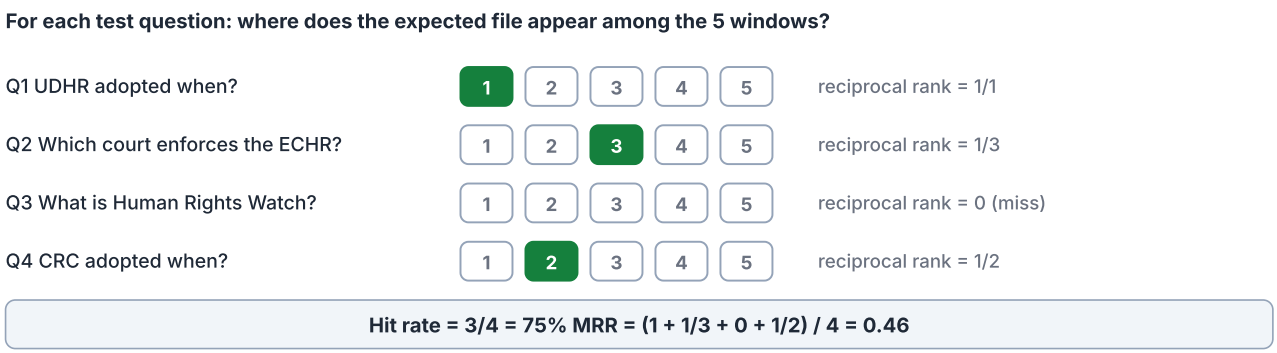
Beyond the Course

IN THIS PHASE

Bonus material covering the gaps the course leaves open. You will measure retrieval quality with numbers instead of gut feeling, add database indexes, combine keyword and vector search (hybrid search), let the LLM re-rank results, and put a small web chat UI on top. Each topic is independent, so pick the ones that interest you.

5.1 Measure before you improve

Every change you make to a RAG system (chunk size, window size, prompt, model, search method) can help or hurt. Without measurements you are guessing. The simplest useful measurement only checks **retrieval**: for a set of test questions where you know which file holds the answer, does that file show up among the 5 windows, and how high up?



Hit rate counts how often the right file appears at all. MRR also rewards putting it first.

- **Hit rate:** the share of questions whose expected file appears among the windows.
- **MRR (mean reciprocal rank):** for each question, score 1 if the right file is first, ½ if second, ⅓ if third, and 0 if it's missing. Then average. MRR rewards putting the right file near the top.

The test set is a small JSON file. The sample targets well-known human-rights articles. **Edit it to match the files you actually downloaded.** The `eval` command skips any question whose file isn't in your corpus and tells you so.

data/eval/questions.json

```
[
  { "question": "When was the Universal Declaration of Human Rights adopted?", "expectedFile":
"Universal_Declaration_of_Human_Rights.txt" },
  { "question": "Which court enforces the European Convention on Human Rights?", "expectedFile":
"European_Convention_on_Human_Rights.txt" },
  { "question": "What does the International Covenant on Civil and Political Rights protect?",
"expectedFile": "International_Covenant_on_Civil_and_Political_Rights.txt" },
  { "question": "What is the role of the UN Human Rights Council?", "expectedFile":
"United_Nations_Human_Rights_Council.txt" },
  { "question": "What does Amnesty International campaign for?", "expectedFile":
"Amnesty_International.txt" },
```

```
  { "question": "What are economic, social and cultural rights?", "expectedFile":
"Economic__social_and_cultural_rights.txt" },
  { "question": "What is Human Rights Watch?", "expectedFile": "Human_Rights_Watch.txt" },
  { "question": "When was the Convention on the Rights of the Child adopted?", "expectedFile":
"Convention_on_the_Rights_of_the_Child.txt" }
]
```

src/Rag.Core/Evaluation/Evaluator.cs

```
using System.Text.Json;
using Rag.Core.Generation;

namespace Rag.Core.Evaluation;

/// <summary>One test question and the file that contains its answer.</summary>
public sealed record EvalQuestion(string Question, string ExpectedFile);

/// <summary>
/// Phase 5: measures retrieval quality so you can tell whether a change actually helped.
///   Hit rate@k: share of questions where the right file is among the k windows.
///   MRR (mean reciprocal rank): rewards putting the right file FIRST (1st = 1, 2nd = 0.5, 3rd =
0.33...).
/// </summary>
public sealed class Evaluator(RagAnswerer rag)
{
    public static List<EvalQuestion> Load(string path) =>
        JsonSerializer.Deserialize<List<EvalQuestion>>(File.ReadAllText(path),
            new JsonSerializerOptions { PropertyNameCaseInsensitive = true }) ?? [];

    public async Task<(double HitRate, double Mrr)> RunAsync(IReadOnlyList<EvalQuestion> questions,
RetrievalMode mode, CancellationToken ct = default)
    {
        int hits = 0;
        double reciprocalRankSum = 0;

        Console.WriteLine($"Mode: {mode}");
        foreach (var q in questions)
        {
            var context = await rag.RetrieveAsync(q.Question, mode, ct);
            int rank = context.Windows.FindIndex(w => w.FileName.Equals(q.ExpectedFile,
StringComparison.OrdinalIgnoreCase)) + 1;

            if (rank > 0) { hits++; reciprocalRankSum += 1.0 / rank; }
            Console.WriteLine($"  {(rank > 0 ? $"rank {rank}" : "MISS  "),-7} {q.Question}");
        }

        double hitRate = questions.Count == 0 ? 0 : (double)hits / questions.Count;
        double mrr = questions.Count == 0 ? 0 : reciprocalRankSum / questions.Count;
        Console.WriteLine($"Hit rate: {hitRate:P0}   MRR: {mrr:F2}   ({questions.Count}
questions)");
        return (hitRate, mrr);
    }
}
```

src/Rag.Cli/Program.cs: EvalAsync()

```
async Task EvalAsync()
{
    var file = Option("--file") ?? Path.Combine("data", "eval", "questions.json");
    var questions = Evaluator.Load(file);

    // A question whose expected file is not in the corpus can never be a hit: warn and skip it.
    var missing = questions.Where(q => !File.Exists(Path.Combine(settings.CorpusFolder,
q.ExpectedFile))).ToList();
```

```
    foreach (var q in missing) Console.WriteLine($"  (skipped: '{q.ExpectedFile}' is not in
{settings.CorpusFolder})");
    questions = questions.Except(missing).ToList();
    if (questions.Count == 0) { Console.WriteLine("No usable questions. Edit the eval file to match
your corpus."); return; }

    await using var db = new RagDbContext(settings);
    var evaluator = new Evaluator(CreateAnswerer(db));

    var results = new List<(RetrievalMode Mode, double HitRate, double Mrr)>();
    var modes = Flag("--rerank")
        ? new[] { RetrievalMode.Vector, RetrievalMode.Hybrid, RetrievalMode.HybridRerank }
        : new[] { RetrievalMode.Vector, RetrievalMode.Hybrid };

    foreach (var mode in modes)
    {
        var (hitRate, mrr) = await evaluator.RunAsync(questions, mode, ct);
        results.Add((mode, hitRate, mrr));
        Console.WriteLine();
    }

    Console.WriteLine("Summary");
    Console.WriteLine("  Mode           Hit rate   MRR");
    foreach (var r in results) Console.WriteLine($"  {r.Mode,-14} {r.HitRate,8:P0}   {r.Mrr:F2}");
}
```

```
dotnet run --project src/Rag.Cli -- eval
```

```
Mode: Vector
  rank 1  When was the Universal Declaration of Human Rights adopted?
  rank 2  Which court enforces the European Convention on Human Rights?
  MISS    What is Human Rights Watch?
  ...
Summary
  Mode           Hit rate   MRR
  Vector              75%   0.58
  Hybrid              88%   0.71
```

(Your numbers will differ. Those above are only for illustration.)

TIP

Twenty to fifty questions give far more reliable numbers than eight. Write them the way real users would ask, including some that use different words from the article. Re-run `eval` after every change, and keep a change only if the numbers improve.

5.2 Indexes: HNSW and full-text

So far every search compares the question with every row (an exact search). That is fine for 10,000 sentences, but slows down as the corpus grows. Two indexes help:

- **HNSW** (Hierarchical Navigable Small World) is a graph index for **approximate** nearest-neighbour search. It's extremely fast, and it occasionally misses a true neighbour. pgvector can index vectors of up to 2,000 dimensions, which our 1,024 fit comfortably (the course's 4,096 would not).

- **GIN on `to_tsvector(content)`** speeds up PostgreSQL's built-in full-text search, which the hybrid search in the next section needs.

src/Rag.Cli/Program.cs: IndexAsync()

```
async Task IndexAsync()
{
    await using var db = new RagDbContext(settings);
    Console.WriteLine("Creating HNSW index on embedding (approximate nearest neighbour search)...");
    await db.Database.ExecuteSqlRawAsync(
        "CREATE INDEX IF NOT EXISTS ix_text_embeddings_hnsw ON text_embeddings USING hnsw (embedding
vector_cosine_ops);", ct);
    Console.WriteLine("Creating GIN index for full-text (keyword) search...");
    await db.Database.ExecuteSqlRawAsync(
        "CREATE INDEX IF NOT EXISTS ix_text_embeddings_fts ON text_embeddings USING gin
(to_tsvector('english', content));", ct);
    Console.WriteLine("Done.");
}
```

```
dotnet run --project src/Rag.Cli -- index
```

WATCH OUT

The `ef_search` trap. With an HNSW index, pgvector returns at most `hnsw.ef_search` rows per query (40 by default), even when you ask for `LIMIT 50`. Your candidate pool would shrink without any error. `Retriever.EnsureSessionSettingsAsync` raises it to 100 for the connection. It opens the connection explicitly, so EF Core keeps using that same connection and the setting sticks:

src/Rag.Core/Retrieval/Retriever.cs: EnsureSessionSettingsAsync

```
await EnsureSessionSettingsAsync(ct);
var q = new Vector(queryVector);
return await db.TextEmbeddings
    .AsNoTracking()
    .OrderBy(e => e.Embedding.CosineDistance(q))
    .Take(k)
    .Select(e => new SearchHit(e.Id, e.FileName, e.SentenceNumber, e.Content,
e.Embedding.CosineDistance(q)))
    .ToListAsync(ct);
    }

    private bool _sessionReady;

    /// <summary>
    /// Once you add an HNSW index (Phase 5), pgvector returns at most hnsw.ef_search rows
    /// (default 40) per query, fewer than our 50 candidates. Raising it per session avoids that.
    /// We open the connection ourselves so EF Core keeps using the same one (and the setting
sticks).
    /// </summary>
    private async Task EnsureSessionSettingsAsync(CancellationToken ct)
    {
if (_sessionReady) return;
await db.Database.OpenConnectionAsync(ct);
int efSearch = Math.Max(100, settings.CandidatePool * 2);   // a plain number, so no SQL-injection
risk
await db.Database.ExecuteSqlRawAsync("SET hnsw.ef_search = " + efSearch, ct);
_sessionReady = true;
    }
```

```
    // ------------------------------------------------------------------ Phase 5

    /// <summary>
    /// Keyword (full-text) search with PostgreSQL's built-in text search.
    /// Great at exact names and rare terms, which embeddings sometimes miss.
    /// </summary>
    public async Task<List<SearchHit>> KeywordSearchAsync(string question, int k, CancellationToken
ct = default)
    {
var rows = await db.Database.SqlQuery<KeywordRow>($"""
    SELECT id AS "Id", file_name AS "FileName", sentence_number AS "SentenceNumber", content AS
"Content",
           ts_rank(to_tsvector('english', content), websearch_to_tsquery('english', {question})) AS
"Rank"
    FROM text_embeddings
    WHERE to_tsvector('english', content) @@ websearch_to_tsquery('english', {question})
    ORDER BY "Rank" DESC
    LIMIT {k}
    """).ToListAsync(ct);

return rows.Select(r => new SearchHit(r.Id, r.FileName, r.SentenceNumber, r.Content,
r.Rank)).ToList();
    }

    /// <summary>Hybrid search: run vector + keyword search and fuse the two rankings with RRF.
</summary>
    public async Task<List<SearchHit>> HybridSearchAsync(string question, float[] queryVector, int
k, CancellationToken ct = default)
    {
var vector = await VectorSearchAsync(queryVector, k, ct);
var keyword = await KeywordSearchAsync(question, k, ct);
return RankFusion.Fuse([vector, keyword]).Take(k).ToList();
    }

    // ------------------------------------------------------------------ Phase 4

    /// <summary>
    /// The full retrieval pipeline:
    /// question -> embedding -> many candidate hits -> merged, non-overlapping windows -> sentence
text.
    /// </summary>
    public async Task<List<ContextWindow>> RetrieveWindowsAsync(
string question, bool hybrid = false, int? maxWindows = null, CancellationToken ct = default)
    {
var queryVector = await embeddings.EmbedQueryAsync(question, ct);

var candidates = hybrid
    ? await HybridSearchAsync(question, queryVector, settings.CandidatePool, ct)
    : await VectorSearchAsync(queryVector, settings.CandidatePool, ct);

var windows = WindowMerger.SelectWindows(candidates, settings.WindowSize, maxWindows ??
settings.MaxWindows);
await LoadSentencesAsync(windows, ct);
return windows;
    }

    /// <summary>Fetches the actual sentences for each window (one small indexed query per window).
</summary>
    public async Task LoadSentencesAsync(IEnumerable<ContextWindow> windows, CancellationToken ct =
default)
    {
foreach (var w in windows)
{
    var sentences = await db.TextEmbeddings
        .AsNoTracking()
        .Where(e => e.FileName == w.FileName && e.SentenceNumber >= w.Start && e.SentenceNumber <=
w.End)
```

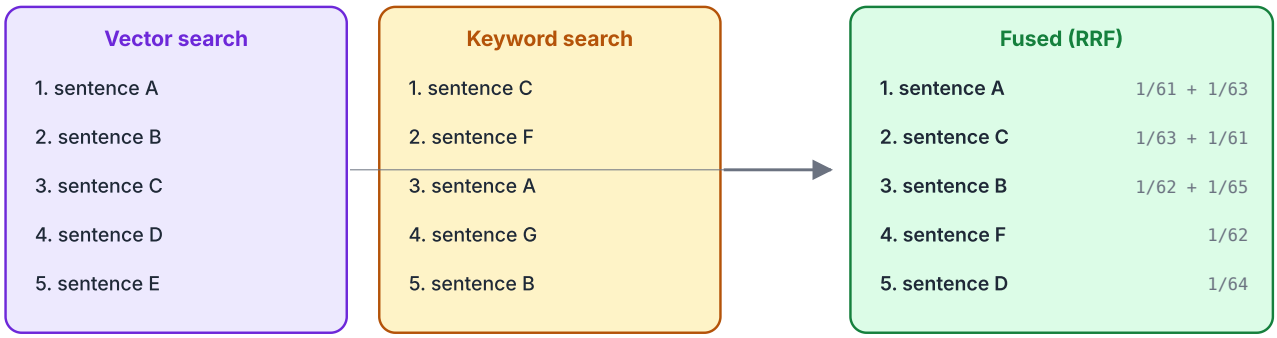
```
        .OrderBy(e => e.SentenceNumber)
        .Select(e => e.Content)
        .ToListAsync(ct);

    w.Sentences.Clear();
    w.Sentences.AddRange(sentences);
}
```

5.3 Hybrid search

Vector search understands meaning but can miss exact terms: a rare name, a treaty number, an acronym. Keyword search is the opposite. **Hybrid search** runs both and merges the two ranked lists.

The merge uses **Reciprocal Rank Fusion (RRF)**. Each sentence scores `1 / (60 + rank)` in every list it appears in, and the scores are added up. A sentence found by both methods rises to the top. RRF only looks at ranks, never at raw scores, so it doesn't matter that cosine distances and keyword ranks are on different scales.



score = Σ 1 / (60 + rank). Found by BOTH methods → rises to the top.

Keyword search catches exact names and numbers; vector search catches meaning.

Two rankings fused into one. Sentences found by both methods win.

src/Rag.Core/Retrieval/RankFusion.cs

```
namespace Rag.Core.Retrieval;

/// <summary>
/// Reciprocal Rank Fusion (RRF): combines several ranked lists into one (Phase 5, hybrid search).
/// score(item) = sum over lists of 1 / (k + rank), where rank starts at 1.
/// Items that rank high in BOTH lists float to the top. k = 60 is the usual default.
/// </summary>
public static class RankFusion
{
    public static List<SearchHit> Fuse(IEnumerable<IReadOnlyList<SearchHit>> rankedLists, int k =
60)
    {
        var scores = new Dictionary<long, double>();
        var byId = new Dictionary<long, SearchHit>();

        foreach (var list in rankedLists)
        {
            for (int i = 0; i < list.Count; i++)
            {
                var hit = list[i];
                scores[hit.Id] = scores.GetValueOrDefault(hit.Id) + 1.0 / (k + i + 1);
                byId.TryAdd(hit.Id, hit);
```

```
            }
        }

        return scores
            .OrderByDescending(kv => kv.Value)
            .Select(kv => byId[kv.Key] with { Score = kv.Value })
            .ToList();
    }
}
```

The keyword search is raw SQL through EF Core's `SqlQuery`. `websearch_to_tsquery` accepts ordinary search-box text, and `ts_rank` scores how well each sentence matches. The interpolated values (`{question}`, `{k}`) become **SQL parameters**, not pasted text, so this is safe from SQL injection:

src/Rag.Core/Retrieval/Retriever.cs: KeywordSearchAsync

```
/// <summary>
/// Keyword (full-text) search with PostgreSQL's built-in text search.
/// Great at exact names and rare terms, which embeddings sometimes miss.
/// </summary>
public async Task<List<SearchHit>> KeywordSearchAsync(string question, int k, CancellationToken ct =
default)
{
    var rows = await db.Database.SqlQuery<KeywordRow>($"""
        SELECT id AS "Id", file_name AS "FileName", sentence_number AS "SentenceNumber", content AS
"Content",
               ts_rank(to_tsvector('english', content), websearch_to_tsquery('english', {question}))
AS "Rank"
        FROM text_embeddings
        WHERE to_tsvector('english', content) @@ websearch_to_tsquery('english', {question})
        ORDER BY "Rank" DESC
        LIMIT {k}
        """).ToListAsync(ct);

    return rows.Select(r => new SearchHit(r.Id, r.FileName, r.SentenceNumber, r.Content,
r.Rank)).ToList();
}
```

src/Rag.Core/Retrieval/Retriever.cs: HybridSearchAsync

```
/// <summary>Hybrid search: run vector + keyword search and fuse the two rankings with RRF.
</summary>
public async Task<List<SearchHit>> HybridSearchAsync(string question, float[] queryVector, int k,
CancellationToken ct = default)
{
    var vector = await VectorSearchAsync(queryVector, k, ct);
    var keyword = await KeywordSearchAsync(question, k, ct);
    return RankFusion.Fuse([vector, keyword]).Take(k).ToList();
}
```

```
dotnet run --project src/Rag.Cli -- ask "What did the 1966 covenants establish?" --mode hybrid
```

5.4 Re-ranking with the LLM

Search ranks by similarity, but *similar* is not always *useful*. A **re-ranker** takes a second, more careful look. Here the base Granite model reads each candidate window and scores from 0 to 10 how well it answers the question. With `--mode rerank`, the answerer fetches twice as many windows (10), re-ranks them and

keeps the best 5.

src/Rag.Core/Generation/LlmReranker.cs

```
using System.Globalization;
using System.Text.RegularExpressions;
using Microsoft.Extensions.AI;
using Rag.Core.Retrieval;

namespace Rag.Core.Generation;

/// <summary>
/// Phase 5: a second, smarter sorting pass. Vector search is fast but rough; here the LLM
/// reads each candidate window and scores how well it answers the question (0-10).
///
/// This is slow on a CPU (one LLM call per window), so it is optional. Dedicated
/// "cross-encoder" re-ranking models do the same job much faster when you have a GPU.
/// </summary>
public sealed partial class LlmReranker(IChatClient baseChat)
{
    public async Task<List<ContextWindow>> RerankAsync(string question, List<ContextWindow> windows,
CancellationToken ct = default)
    {
        var scored = new List<(ContextWindow Window, double Score)>();
        foreach (var w in windows)
        {
            var prompt = $"""
                Rate how useful the passage is for answering the question.
                Reply with a single number from 0 (useless) to 10 (fully answers it). No other text.

                Question: {question}

                Passage: {string.Join(" ", w.Sentences)}
                """;

            var response = await baseChat.GetResponseAsync(prompt, new ChatOptions { Temperature = 0
}, ct);
            scored.Add((w, ParseScore(response.Text)));
        }

        // Stable sort: equal scores keep their original (vector) order.
        return scored.OrderByDescending(s => s.Score).Select(s => s.Window).ToList();
    }

    private static double ParseScore(string text)
    {
        var m = FirstNumber().Match(text);
        return m.Success ? double.Parse(m.Value, CultureInfo.InvariantCulture) : 0;
    }

    [GeneratedRegex(@"\d+(\.\d+)?")]
    private static partial Regex FirstNumber();
}
```

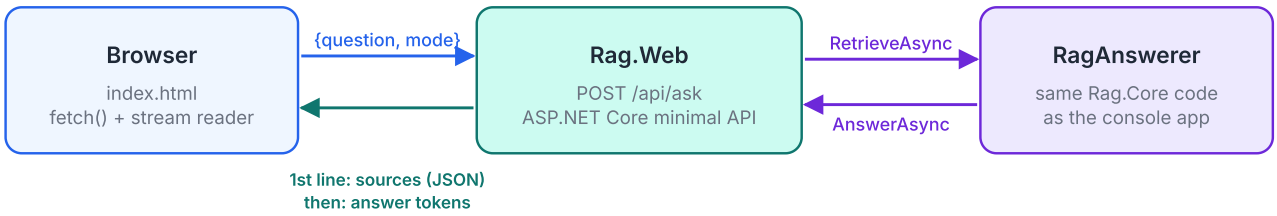
ON YOUR LAPTOP

Re-ranking makes one LLM call per window: 10 extra calls per question. On your CPU that adds noticeable time to every answer, so treat it as an experiment to measure with `rag eval --rerank` rather than a default. Dedicated “cross-encoder” re-ranking models do the same job far faster, and are worth exploring once you have a stronger GPU.

Note that the re-ranker uses the **base** model (`granite4.1:3b`), not `rag-granite`. Our RAG system prompt ("answer only from context...") would interfere with a scoring task.

5.5 A web chat UI

The same `Rag.Core` library can power a web page. ASP.NET Core's minimal APIs need only a few lines. The one interesting part is **streaming**: the endpoint writes the source list as the first line, then writes and *flushes* each token as it arrives, so the browser shows the answer while it's being generated.



The browser posts the question. The server streams back the sources, then the answer tokens.

src/Rag.Web/Rag.Web.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\Rag.Core\Rag.Core.csproj" />
  </ItemGroup>

</Project>
```

src/Rag.Web/Program.cs

```
using System.Text.Json;
using Rag.Core;
using Rag.Core.Ai;
using Rag.Core.Data;
using Rag.Core.Generation;
using Rag.Core.Retrieval;

// Phase 5: a tiny web chat UI on top of the same Rag.Core library the console app uses.
// Run:  dotnet run --project src/Rag.Web     then open the URL it prints (e.g.
http://localhost:5000)

var builder = WebApplication.CreateBuilder(args);

var settings = new RagSettings();
builder.Services.AddSingleton(settings);
builder.Services.AddSingleton(_ => new
EmbeddingService(AiClients.CreateEmbeddingGenerator(settings)));
builder.Services.AddScoped<RagDbContext>();
builder.Services.AddScoped<Retriever>();
```

```
builder.Services.AddScoped(sp => new RagAnswerer(
    sp.GetRequiredService<Retriever>(),
    AiClients.CreateChatClient(settings),
    settings,
    new LlmReranker(AiClients.CreateChatClient(settings, settings.BaseChatModel))));

var app = builder.Build();
app.UseDefaultFiles();   // serves wwwroot/index.html at "/"
app.UseStaticFiles();

// POST /api/ask  { "question": "...", "mode": "vector" | "hybrid" | "rerank" }
// Response (streamed text): first line = JSON array of sources, then the answer tokens as they
arrive.
app.MapPost("/api/ask", async (AskRequest request, RagAnswerer rag, HttpResponse response,
CancellationToken ct) =>
{
    var mode = request.Mode?.ToLowerInvariant() switch
    {
        "hybrid" => RetrievalMode.Hybrid,
        "rerank" => RetrievalMode.HybridRerank,
        _ => RetrievalMode.Vector,
    };

    var context = await rag.RetrieveAsync(request.Question, mode, ct);

    response.ContentType = "text/plain; charset=utf-8";
    await response.WriteAsync(JsonSerializer.Serialize(context.Sources) + "\n", ct);
    await response.Body.FlushAsync(ct);

    await foreach (var token in rag.AnswerAsync(context, ct))
    {
        await response.WriteAsync(token, ct);
        await response.Body.FlushAsync(ct);   // push each token to the browser immediately
    }
});

app.Run();

public sealed record AskRequest(string Question, string? Mode);
```

The page reads the stream with `fetch()` and a `ReadableStream` reader. The first line is parsed as JSON (the sources). Everything after it is appended to the answer as it arrives.

src/Rag.Web/wwwroot/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>RAG from scratch: chat</title>
  <style>
    body { font-family: system-ui, sans-serif; max-width: 820px; margin: 2rem auto; padding: 0 1rem;
color: #1f2933; }
    h1 { font-size: 1.4rem; }
    form { display: flex; gap: .5rem; }
    input[type=text] { flex: 1; padding: .6rem; font-size: 1rem; border: 1px solid #cbd2d9; border-
radius: 6px; }
    button, select { padding: .6rem 1rem; font-size: 1rem; border-radius: 6px; border: 1px solid
#cbd2d9; }
    button { background: #2563eb; color: white; border: none; cursor: pointer; }
    button:disabled { background: #93b4f5; }
    .turn { margin-top: 1.5rem; }
    .q { font-weight: 600; }
    .a { white-space: pre-wrap; line-height: 1.5; margin-top: .4rem; }
```

```
    .sources { font-size: .85rem; color: #52606d; margin-top: .5rem; }
  </style>
</head>
<body>
  <h1>Ask your documents</h1>
  <form id="f">
    <input id="q" type="text" placeholder="e.g. Is the Dominican Republic a member of the United
Nations?" autofocus>
    <select id="mode">
      <option value="vector">vector</option>
      <option value="hybrid">hybrid</option>
      <option value="rerank">hybrid + rerank</option>
    </select>
    <button id="go">Ask</button>
  </form>
  <div id="log"></div>

<script>
const form = document.getElementById('f'), log = document.getElementById('log'), go =
document.getElementById('go');

form.addEventListener('submit', async e => {
  e.preventDefault();
  const question = document.getElementById('q').value.trim();
  if (!question) return;
  go.disabled = true;

  const turn = document.createElement('div'); turn.className = 'turn';
  turn.innerHTML = '<div class="q"></div><div class="a">Searching...</div><div class="sources">
</div>';
  turn.querySelector('.q').textContent = question;
  log.prepend(turn);
  const answer = turn.querySelector('.a'), sources = turn.querySelector('.sources');

  try {
    const res = await fetch('/api/ask', {
      method: 'POST', headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ question, mode: document.getElementById('mode').value })
    });
    const reader = res.body.getReader(), decoder = new TextDecoder();
    let buffer = '', gotSources = false;
    while (true) {
      const { done, value } = await reader.read();
      if (done) break;
      buffer += decoder.decode(value, { stream: true });
      if (!gotSources) {                    // first line = JSON list of sources
        const nl = buffer.indexOf('\n');
        if (nl < 0) continue;
        sources.textContent = JSON.parse(buffer.slice(0, nl)).join('  ·  ');
        buffer = buffer.slice(nl + 1); gotSources = true; answer.textContent = '';
      }
      answer.textContent += buffer; buffer = '';
    }
  } catch (err) {
    answer.textContent = 'Error: ' + err;
  } finally {
    go.disabled = false;
  }
});
</script>
</body>
</html>
```

```
dotnet run --project src/Rag.Web
# Now listening on: http://localhost:5xxx   <- open this in your browser
```

5.6 Where to go next

You now understand every layer of a RAG system, because you built each one by hand. Here are some next steps, roughly in order of payoff:

Idea	What it gives you	Where to start
Better chunking	Chunks of ~200–400 tokens with overlap often beat single sentences	Add a `TokenChunker` next to `SentenceSplitter`, then compare the two with `rag eval`
Bigger test set	Trustworthy measurements	30–50 questions, including tricky paraphrases
Larger models	Better answers and citations	`granite4.1:8b` for chat, `qwen3-embedding:4b` for embeddings (2,560 dimensions; re-ingest needed)
Microsoft.Extensions.VectorData	A standard vector-store abstraction (PostgreSQL, SQLite, Qdrant, ...)	Replace `RagDbContext` searches with its PgVector connector
Microsoft.Extensions.DataIngestion	Ready-made readers, chunkers and writers	Compare its pipeline with your `Ingestor` (preview package)
Conversation memory	Follow-up questions ("and in Germany?")	Rewrite the follow-up into a standalone question with the LLM before retrieval
Answer evaluation	Measure faithfulness, not just retrieval	Ask a model to check that each cited claim appears in its source
Agents and tools	Multi-step reasoning, calling APIs	Semantic Kernel or the Microsoft Agent Framework, using `IChatClient`

CHECKPOINT

- `rag eval` prints a summary table, and you can explain hit rate and MRR.
- `rag index` ran, and searches still return 50 candidates (thanks to `ef_search`).
- You have compared `--mode vector` and `--mode hybrid` on your own questions.
- The web UI streams answers with sources.

APPENDICES

Reference

A. Command reference

Run every command from the solution folder. To save typing, add a PowerShell function to your profile (`notepad $PROFILE`):

```
function rag { dotnet run --project "$HOME\source\RagFromScratch\src\Rag.Cli" -- @args }
```

(Adjust the path to wherever you put the solution.) After that, `rag ask "..."` works from anywhere.

Command	Options	Phase
`rag selftest`		any
`rag check`		2
`rag similarity "s1" "s2" ...`		2
`rag corpus`	`--topic "human rights"` `--max 100`	2
`rag extract [file or folder]`	default folder `data/my_docs`	2
`rag ingest`	`--reset` (empty the table first), `--max-files N`	3
`rag search "query"`	`--k 5`	3
`rag ask "question"`	`--show-prompt`, `--mode vector | hybrid | rerank`	4–5
`rag index`		5
`rag eval`	`--file path.json`, `--rerank`	5

Ollama and Docker commands used in this book:

Command	Purpose
`ollama pull <model>`	Download a model
`ollama list` / `ollama ps`	Installed models / loaded models, and CPU vs GPU
`ollama run <model>`	Chat in the terminal (`/bye` to exit)
`ollama create rag-granite -f Modelfile`	Build the custom model (re-run after editing the Modelfile)
`ollama show --modelfile <model>`	Show a model's template, parameters and system prompt
`docker compose up -d` / `down` / `down -v`	Start / stop / stop and delete the database

Command	Purpose
`docker exec -it rag-postgres psql -U postgres -d text_embeddings`	SQL prompt inside the database

B. Troubleshooting

Problem	Fix
`Could not reach Ollama at http://localhost:11434`	Start Ollama from the Start menu (look for the llama tray icon), or run `ollama serve`
`model "rag-granite" not found`	Run `ollama create rag-granite -f Modelfile` from the solution folder
`check` says the embedding has a different number of dimensions	`RAG_EMBED_DIMS` doesn't match the model. Fix it and re-ingest with `--reset`
`expected 1024 dimensions, not 768` when inserting	The table was created for another model. Run `docker compose down -v`, `docker compose up -d`, then ingest again
`Npgsql ... 28P01 password authentication failed`	Check `RAG_PG` matches `compose.yaml` (user `postgres`, password `postgres`)
`type "vector" does not exist`	The database was created before `init.sql` was mounted. Run `docker compose down -v` and `up -d`, or run `CREATE EXTENSION vector;` in psql
Port 5432 already in use	Another PostgreSQL is running. Stop it, or change the port to `"5433:5432"` in `compose.yaml` and in `RAG_PG`
Everything is very slow	`ollama ps`: if it shows 100% CPU, that's expected on this laptop. Close other heavy apps, keep models on the SSD, use 3B models
GPU not used	Update the NVIDIA driver to 570+, restart Ollama, check with `nvidia-smi` and `ollama ps`
Requests time out	Our `HttpClient` timeout is 10 minutes (`AiClients.cs`). For very slow runs, raise it or lower `EmbeddingBatchSize`
Wikipedia download fails	Check your internet connection. One failed article is skipped and the run continues. Re-run `corpus` to fill gaps
`dotnet build` reports an API difference in a package	Packages evolve. Pin the versions from the `.csproj` (`dotnet add package X --version Y`), or check the package's release notes for the renamed member

C. Complete console app listings

`Program.cs` holds all the commands. Each phase discussed its own functions. Here is the whole file:

src/Rag.Cli/Program.cs

```
using System.Diagnostics;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.AI;
using Rag.Core;
using Rag.Core.Ai;
using Rag.Core.Corpus;
using Rag.Core.Data;
using Rag.Core.Evaluation;
using Rag.Core.Generation;
using Rag.Core.Ingestion;
using Rag.Core.Retrieval;
using Rag.Core.Text;

// ------------------------------------------------------------------------------------------
//  rag <command> [arguments]          (run from the repository root: dotnet run --project
src/Rag.Cli -- <command>)
//
//  Phase 1   check                       Is Ollama + Postgres reachable? Are the models there?
//            similarity "a" "b" ["c"..]  Embed sentences and print how similar they are
//  Phase 2   corpus [--topic t] [--max n] Download Wikipedia articles into data/all_articles
//            extract <file|folder>       Turn PDF / DOCX / PPTX into .txt in data/all_articles
//  Phase 3   ingest [--reset] [--max-files n]   Split, embed and store every .txt file
//            search "query" [--k n]      Show the raw nearest sentences
//  Phase 4   ask "question" [--show-prompt] [--mode vector|hybrid|rerank]
//  Phase 5   index                       Create HNSW (vector) + GIN (full-text) indexes
//            eval [--file path]          Hit rate / MRR for vector vs hybrid (vs rerank with --
rerank)
//  Any       selftest                    Run the offline unit checks (no Ollama/DB needed)
// ------------------------------------------------------------------------------------------

var settings = new RagSettings();
using var cts = new CancellationTokenSource();
Console.CancelKeyPress += (_, e) => { e.Cancel = true; cts.Cancel(); };
var ct = cts.Token;

if (args.Length == 0) { PrintHelp(); return 0; }

try
{
    switch (args[0].ToLowerInvariant())
    {
        case "check": await CheckAsync(); break;
        case "similarity": await SimilarityAsync(args[1..]); break;
        case "corpus": await CorpusAsync(); break;
        case "extract": Extract(); break;
        case "ingest": await IngestAsync(); break;
        case "search": await SearchAsync(); break;
        case "ask": await AskAsync(); break;
        case "index": await IndexAsync(); break;
        case "eval": await EvalAsync(); break;
        case "selftest": return SelfTest.Run();
        default: PrintHelp(); return 1;
    }
    return 0;
}
catch (OperationCanceledException)
{
    Console.WriteLine("\nCancelled.");
    return 130;
}
catch (ArgumentException ex)
{
    Console.Error.WriteLine(ex.Message);
    return 2;
```

```
}
catch (HttpRequestException ex)
{
    Console.Error.WriteLine($"Could not reach Ollama at {settings.OllamaUrl}: {ex.Message}");
    Console.Error.WriteLine("Is Ollama running? (Windows: check the tray icon, or run 'ollama
serve')");
    return 1;
}

// ================================================================================ Phase 1

async Task CheckAsync()
{
    Console.WriteLine($"Ollama:     {settings.OllamaUrl}");
    var chat = AiClients.CreateChatClient(settings);
    var sw = Stopwatch.StartNew();
    var reply = await chat.GetResponseAsync("Reply with exactly: OK", cancellationToken: ct);
    Console.WriteLine($"  chat model  '{settings.ChatModel}' replied '{reply.Text.Trim()}' in
{sw.Elapsed.TotalSeconds:F1}s");

    var embeddings = new EmbeddingService(AiClients.CreateEmbeddingGenerator(settings));
    var vector = await embeddings.EmbedRawAsync("hello world", ct);
    Console.WriteLine($"  embed model '{settings.EmbeddingModel}' -> {vector.Length} dimensions" +
                      (vector.Length == settings.EmbeddingDimensions ? "" : $"  (!) settings expect
{settings.EmbeddingDimensions}"));

    await using var db = new RagDbContext(settings);
    Console.WriteLine($"PostgreSQL: {(await db.Database.CanConnectAsync(ct) ? "connected" : "NOT
reachable")}");
}

async Task SimilarityAsync(string[] sentences)
{
    if (sentences.Length < 2) { Console.WriteLine("Give at least two sentences in quotes."); return;
}
    var embeddings = new EmbeddingService(AiClients.CreateEmbeddingGenerator(settings));
    var vectors = new List<float[]>();
    foreach (var s in sentences) vectors.Add(await embeddings.EmbedRawAsync(s, ct));

    Console.WriteLine($"Each sentence became a vector of {vectors[0].Length} numbers. First 5 of #1:
" +
                      string.Join(", ", vectors[0].Take(5).Select(v => v.ToString("F3"))) + ",
...\n");
    Console.WriteLine("Cosine similarity (1.00 = same meaning, ~0 = unrelated):");
    for (int i = 0; i < sentences.Length; i++)
        for (int j = i + 1; j < sentences.Length; j++)
            Console.WriteLine($"  {VectorMath.CosineSimilarity(vectors[i], vectors[j]):F2}  \"
{sentences[i]}\"  vs  \"{sentences[j]}\"");
}

// ================================================================================ Phase 2

async Task CorpusAsync()
{
    var topic = Option("--topic") ?? "human rights";
    var max = int.Parse(Option("--max") ?? "100");
    using var generator = new WikipediaCorpusGenerator();
    await generator.GenerateAsync(topic, max, settings.CorpusFolder, ct);
}

void Extract()
{
    var input = args.Length > 1 ? args[1] : Path.Combine("data", "my_docs");
    Console.WriteLine($"Extracting '{input}' -> '{settings.CorpusFolder}'");
    var n = DocumentTextExtractor.ExtractToFolder(input, settings.CorpusFolder);
    Console.WriteLine($"{n} file(s) extracted.");
}
```

```
// ============================================================================ Phase 3

async Task IngestAsync()
{
    await using var db = new RagDbContext(settings);
    var embeddings = new EmbeddingService(AiClients.CreateEmbeddingGenerator(settings));
    var maxFiles = Option("--max-files") is { } m ? int.Parse(m) : (int?)null;
    await new Ingestor(db, embeddings, settings).IngestFolderAsync(settings.CorpusFolder, Flag("--
reset"), maxFiles, ct);
}

async Task SearchAsync()
{
    var query = RequireText("search");
    var k = int.Parse(Option("--k") ?? "5");
    await using var db = new RagDbContext(settings);
    var embeddings = new EmbeddingService(AiClients.CreateEmbeddingGenerator(settings));
    var retriever = new Retriever(db, embeddings, settings);

    var hits = await retriever.VectorSearchAsync(await embeddings.EmbedQueryAsync(query, ct), k,
ct);
    Console.WriteLine($"Top {hits.Count} sentences for: {query}\n");
    foreach (var h in hits)
        Console.WriteLine($"  {h.Score:F3}  {h.FileName} #{h.SentenceNumber}\n
{h.Content}\n");
}

// ============================================================================ Phase 4

async Task AskAsync()
{
    var question = RequireText("ask");
    await using var db = new RagDbContext(settings);
    var rag = CreateAnswerer(db);

    var sw = Stopwatch.StartNew();
    var context = await rag.RetrieveAsync(question, ParseMode(), ct);
    Console.WriteLine($"Retrieved {context.Windows.Count} context window(s) in
{sw.Elapsed.TotalSeconds:F1}s:");
    foreach (var s in context.Sources) Console.WriteLine("  " + s);

    if (Flag("--show-prompt"))
        Console.WriteLine($"\n----- PROMPT SENT TO THE LLM -----\n{context.Prompt}\n----------------
------------------");

    Console.WriteLine("\nAnswer:");
    sw.Restart();
    await foreach (var token in rag.AnswerAsync(context, ct)) Console.Write(token);
    Console.WriteLine($"\n\n({sw.Elapsed.TotalSeconds:F1}s)");
}

// ============================================================================ Phase 5

async Task IndexAsync()
{
    await using var db = new RagDbContext(settings);
    Console.WriteLine("Creating HNSW index on embedding (approximate nearest neighbour search)...");
    await db.Database.ExecuteSqlRawAsync(
        "CREATE INDEX IF NOT EXISTS ix_text_embeddings_hnsw ON text_embeddings USING hnsw (embedding
vector_cosine_ops);", ct);
    Console.WriteLine("Creating GIN index for full-text (keyword) search...");
    await db.Database.ExecuteSqlRawAsync(
        "CREATE INDEX IF NOT EXISTS ix_text_embeddings_fts ON text_embeddings USING gin
(to_tsvector('english', content));", ct);
    Console.WriteLine("Done.");
}
```

```
async Task EvalAsync()
{
    var file = Option("--file") ?? Path.Combine("data", "eval", "questions.json");
    var questions = Evaluator.Load(file);

    // A question whose expected file is not in the corpus can never be a hit: warn and skip it.
    var missing = questions.Where(q => !File.Exists(Path.Combine(settings.CorpusFolder,
q.ExpectedFile))).ToList();
    foreach (var q in missing) Console.WriteLine($"  (skipped: '{q.ExpectedFile}' is not in
{settings.CorpusFolder})");
    questions = questions.Except(missing).ToList();
    if (questions.Count == 0) { Console.WriteLine("No usable questions. Edit the eval file to match
your corpus."); return; }

    await using var db = new RagDbContext(settings);
    var evaluator = new Evaluator(CreateAnswerer(db));

    var results = new List<(RetrievalMode Mode, double HitRate, double Mrr)>();
    var modes = Flag("--rerank")
        ? new[] { RetrievalMode.Vector, RetrievalMode.Hybrid, RetrievalMode.HybridRerank }
        : new[] { RetrievalMode.Vector, RetrievalMode.Hybrid };

    foreach (var mode in modes)
    {
        var (hitRate, mrr) = await evaluator.RunAsync(questions, mode, ct);
        results.Add((mode, hitRate, mrr));
        Console.WriteLine();
    }

    Console.WriteLine("Summary");
    Console.WriteLine("  Mode           Hit rate   MRR");
    foreach (var r in results) Console.WriteLine($"  {r.Mode,-14} {r.HitRate,8:P0}   {r.Mrr:F2}");
}

// ========================================================================== helpers

RagAnswerer CreateAnswerer(RagDbContext db)
{
    var embeddings = new EmbeddingService(AiClients.CreateEmbeddingGenerator(settings));
    var retriever = new Retriever(db, embeddings, settings);
    var reranker = new LlmReranker(AiClients.CreateChatClient(settings, settings.BaseChatModel));
    return new RagAnswerer(retriever, AiClients.CreateChatClient(settings), settings, reranker);
}

RetrievalMode ParseMode() => (Option("--mode") ?? "vector").ToLowerInvariant() switch
{
    "hybrid" => RetrievalMode.Hybrid,
    "rerank" => RetrievalMode.HybridRerank,
    _ => RetrievalMode.Vector,
};

string RequireText(string command)
{
    var text = args.Skip(1).FirstOrDefault(a => !a.StartsWith("--"));
    if (string.IsNullOrWhiteSpace(text)) throw new ArgumentException($"Usage: rag {command} \"your
text\"");
    return text;
}

string? Option(string name)
{
    int i = Array.IndexOf(args, name);
    return i >= 0 && i + 1 < args.Length ? args[i + 1] : null;
}

bool Flag(string name) => args.Contains(name);
```

```
void PrintHelp() => Console.WriteLine("""
    RAG from scratch: commands
      check                              test Ollama + PostgreSQL connections
      similarity "s1" "s2" ["s3" ...]    compare sentence meanings
      corpus [--topic "human rights"] [--max 100]
      extract [file|folder]              PDF/DOCX/PPTX -> .txt (default folder: data/my_docs)
      ingest [--reset] [--max-files N]
      search "query" [--k 5]
      ask "question" [--show-prompt] [--mode vector|hybrid|rerank]
      index                              create HNSW + full-text indexes
      eval [--file data/eval/questions.json] [--rerank]
      selftest                           offline logic checks
    """);
```

`SelfTest.cs` contains the offline checks run by `rag selftest`:

src/Rag.Cli/SelfTest.cs

```
using Rag.Core.Retrieval;
using Rag.Core.Text;

/// <summary>
/// Offline checks for the pure logic (no Ollama, no database needed).
/// Run with:  dotnet run --project src/Rag.Cli -- selftest
/// </summary>
public static class SelfTest
{
    private static int _failures;

    public static int Run()
    {
        SentenceSplitting();
        CosineMaths();
        WindowMerging();
        Fusion();

        Console.WriteLine(_failures == 0 ? "\nAll checks passed." : $"\n{_failures} check(s)
FAILED.");
        return _failures == 0 ? 0 : 1;
    }

    private static void SentenceSplitting()
    {
        Console.WriteLine("Sentence splitting");
        var s = SentenceSplitter.Split(
            "Dr. Smith moved to the U.S. in 1990. He earned $3.5 million! Was it worth it? " +
            "J. R. R. Tolkien wrote books.\n\n== History ==\nThe treaty was signed in 1948.");
        Check(s.Count == 5, $"5 sentences expected, got {s.Count}: {string.Join(" | ", s)}");
        Check(s[0] == "Dr. Smith moved to the U.S. in 1990.", $"abbreviations kept together:
'{s[0]}'");
        Check(s[3].StartsWith("J. R. R. Tolkien"), "initials kept together");
        Check(!s.Any(x => x.Contains("==")), "headings dropped");

        var wrapped = SentenceSplitter.Split("This sentence was hard\nwrapped across two lines. The
next sentence is here.");
        Check(wrapped.Count == 2 && wrapped[0] == "This sentence was hard wrapped across two
lines.", "wrapped lines re-joined");
    }

    private static void CosineMaths()
    {
        Console.WriteLine("Cosine similarity");
        Check(Math.Abs(VectorMath.CosineSimilarity([1, 0], [1, 0]) - 1) < 1e-9, "same direction =
1");
```

```
        Check(Math.Abs(VectorMath.CosineSimilarity([1, 0], [0, 1])) < 1e-9, "perpendicular = 0");
        Check(Math.Abs(VectorMath.CosineSimilarity([1, 0], [-1, 0]) + 1) < 1e-9, "opposite = -1");
        Check(Math.Abs(VectorMath.CosineSimilarity([1, 2, 3], [2, 4, 6]) - 1) < 1e-9, "length does
not matter, only direction");
    }

    private static void WindowMerging()
    {
        Console.WriteLine("Context window merging");
        static SearchHit Hit(string file, int n) => new(n, file, n, $"s{n}", 0);

        // The course's own example: hits at 4 and 8, window +/-2 -> [2..6] and [6..10] share
sentence 6.
        var w = WindowMerger.SelectWindows([Hit("a.txt", 4), Hit("a.txt", 8)], windowSize: 2,
maxWindows: 5);
        Check(w.Count == 1 && w[0].Start == 2 && w[0].End == 10, $"4 and 8 merge into one window
[2..10], got {string.Join(", ", w)}");

        // Touching windows ([2..6] and [7..11]) are merged too, so no gap and no duplicate.
        w = WindowMerger.SelectWindows([Hit("a.txt", 4), Hit("a.txt", 9)], 2, 5);
        Check(w.Count == 1, "adjacent windows merge");

        // Far apart = separate windows.
        w = WindowMerger.SelectWindows([Hit("a.txt", 4), Hit("a.txt", 20)], 2, 5);
        Check(w.Count == 2, "distant hits stay separate");

        // Same sentence numbers in DIFFERENT files never merge.
        w = WindowMerger.SelectWindows([Hit("a.txt", 4), Hit("b.txt", 4)], 2, 5);
        Check(w.Count == 2, "different files never merge");

        // A later hit can bridge two existing windows.
        w = WindowMerger.SelectWindows([Hit("a.txt", 0), Hit("a.txt", 12), Hit("a.txt", 6)], 3, 5);
        Check(w.Count == 1 && w[0].Start == 0 && w[0].End == 15, $"bridging hit merges all three,
got {string.Join(", ", w)}");

        // Stops at maxWindows distinct windows; window start never goes below 0.
        var many = Enumerable.Range(0, 10).Select(i => Hit($"f{i}.txt", 1));
        w = WindowMerger.SelectWindows(many, 3, 5);
        Check(w.Count == 5 && w.All(x => x.Start == 0), "max 5 windows, start clamped at 0");
    }

    private static void Fusion()
    {
        Console.WriteLine("Reciprocal rank fusion");
        static SearchHit Hit(long id) => new(id, "f.txt", (int)id, "", 0);
        var vector = new List<SearchHit> { Hit(1), Hit(2), Hit(3) };
        var keyword = new List<SearchHit> { Hit(3), Hit(4), Hit(1) };
        var fused = RankFusion.Fuse([vector, keyword]);
        Check(fused[0].Id == 1 && fused[1].Id == 3, $"items high in both lists win, got
{string.Join(",", fused.Select(f => f.Id))}");
        Check(fused.Count == 4, "union of both lists");
    }

    private static void Check(bool ok, string description)
    {
        Console.WriteLine($"  {(ok ? "PASS" : "FAIL")}  {description}");
        if (!ok) _failures++;
    }
}
```

D. Python course to .NET mapping

Python course file / concept	This book	Notes
Codespaces / dev container	Local Windows + Docker Desktop	Free, faster disk, GPU possible
`ollama serve` in a terminal	Ollama Windows app	Starts automatically
`modelfile` (`FROM deepseek-r1`, later `gemma3:12b`)	`Modelfile` (`FROM granite4.1:3b`)	Apache 2.0, no thinking mode
`ollama_functions.py` (`embed`, `chat`)	`AiClients.cs`, `EmbeddingService.cs`	`IChatClient` / `IEmbeddingGenerator`
`generate_corpus.py` (`wikipedia` package)	`WikipediaCorpusGenerator.cs`	MediaWiki API, cleans reference sections
`extract_text.py` (Docling)	`DocumentTextExtractor.cs`	PdfPig + Open XML; output is actually ingested
apt install Postgres, `pg_hba.conf` edits	`compose.yaml` + `db/init.sql`	One command
`database_connect_embeddings.py` (SQLAlchemy)	`TextEmbedding.cs`, `RagDbContext.cs`	EF Core + Pgvector
NLTK `sent_tokenize`	`SentenceSplitter.cs`	Rule-based, tested
`populate_vector_db.py`	`Ingestor.cs`	Resumable, no duplicates, stores file and position
sentence-transformers + SFR-Embedding-Mistral	`qwen3-embedding:0.6b` via Ollama	1,024 dims, query instruction
`pull_db_content.py`	`Retriever.VectorSearchAsync`	Same SQL: `ORDER BY embedding <=> q`
`prepare_content.py` (3 grouping helpers)	`WindowMerger.cs`	One loop, off-by-one fixed
`run_rag.py`	`PromptBuilder.cs`, `RagAnswerer.cs`, `rag ask`	Clean context, citations, streaming
`del model; gc.collect()` "memory leak"	Not needed	The model lives in Ollama, not in our process
`screen` for long runs	Not needed	Resumable ingestion
(not covered)	`Evaluator.cs`, `RankFusion.cs`, `LlmReranker.cs`, `Rag.Web`	Phase 5

Course issues fixed in this book

In the course	Effect	Fix here
Overlap check compares hits with `w` instead of `2w`	Overlapping windows not merged; duplicate sentences	`WindowMerger`, tested in `selftest`

In the course	Effect	Fix here
Upload loop doesn't store `file_name` / `sentence_number`	Neighbour lookup can't work	Always stored, plus an index
Neighbours fetched by row `id`	Wrong if rows weren't inserted in order	Filter by file name + sentence number
Context passed as raw tuples	Noisy prompt; no checkable citations	`[Source n]` blocks + citation rule
Embedding model reloaded on every query	Slow, and needed a "memory leak" workaround	Long-lived `IEmbeddingGenerator` client
Re-running ingestion duplicates rows	Duplicated search results	Already-ingested files are skipped; `--reset`
Query and documents embedded the same way	Weaker retrieval with Qwen3-Embedding	Instruction prefix on queries only
Docling output never ingested	Own documents unusable	Extractor writes `.txt` into the corpus folder
Gemma (non-OSI licence), SFR-Embedding (non-commercial)	Licence restrictions	Granite 4.1 and Qwen3-Embedding (Apache 2.0)

E. Glossary

Term	Meaning
Chunk	A piece of a document that gets its own embedding (here: one sentence)
Context window (model)	The maximum number of tokens the model can read and write at once (`num_ctx`)
Context window (retrieval)	The matching sentence plus its neighbours, sent to the model as one passage
Corpus	All the documents the RAG system can answer from
Cosine similarity / distance	How closely two vectors point the same way (1 = same) / `1 − similarity` (0 = same)
Embedding	A vector of numbers representing the meaning of a text
Few-shot prompting	Showing the model example inputs and outputs inside the prompt
GIN index	PostgreSQL index type used for full-text search
Hallucination	A fluent but false answer that isn't supported by any source
HNSW	Graph-based index for fast approximate nearest-neighbour search
Hybrid search	Combining keyword and vector search results

Term	Meaning
Inference	Using a trained model to produce output (as opposed to training it)
LLM	Large language model: predicts the next token, over and over
Modelfile	Ollama's recipe for a custom model: base model, parameters, system prompt, examples
MRR	Mean reciprocal rank: an evaluation score that rewards ranking the right result first
pgvector	PostgreSQL extension that adds the `vector` type and distance operators
Quantization	Storing model weights with fewer bits to save memory
RAG	Retrieval-augmented generation: retrieve passages, add them to the prompt, generate an answer
Re-ranking	A second, more careful ordering of search results
RRF	Reciprocal rank fusion: merges rankings by summing `1 / (k + rank)`
Seed	Fixed starting value for random numbers, which makes output repeatable
System prompt	Standing instructions given to the model before the conversation
Temperature	How random the token choice is (0 = always the most likely token)
Token	A word or word piece; the unit LLMs read and write
Vector database	A database that stores vectors and finds the nearest ones quickly

F. Licences of everything used

Component	Licence	Notes
.NET 10 SDK, EF Core, Microsoft.Extensions.AI	MIT	
Ollama, llama.cpp	MIT	Use local models only. Ollama's cloud models are a paid service
Granite 4.1 (IBM)	Apache 2.0	Commercial use allowed
Qwen3-Embedding (Alibaba)	Apache 2.0	Commercial use allowed
OllamaSharp	MIT	
PostgreSQL, pgvector, Npgsql	PostgreSQL Licence	

Component	Licence	Notes
Pgvector.EntityFrameworkCore	MIT	
PdfPig	Apache 2.0	
Open XML SDK	MIT	
Wikipedia text	CC BY-SA	Credit Wikipedia if you redistribute the text
VSCodium, DotRush	MIT	
Docker Desktop	Proprietary, free tier	Free for personal use, education and small businesses. Larger companies need a subscription, or can use Podman or Rancher Desktop instead

Model licences can change between versions. Check each model's page on ollama.com before commercial use.